1. **Problem Description -:** Given an array Arr[ ] of N integers and a positive integer K. The task is to cyclically rotate the array clockwise by K.
   Note : Keep the first of the array unaltered.
   Example 1:
   - 5 - Value of N
   - {10, 20, 30, 40, 50} -Element of Arr[ ]
   - 2 -Value of K

   Output :
   40 50 10 20 30

   Example 2:
   - 4 - of N
   - {10, 20, 30, 40} - Element of Arr[]
   - 1 - Value of K

   Output :
   40 10 20 30

   <u>JAVA</u>

```
import java.util.*;
public class Main
{
   static int[] rotate(int nums[], int n, int k) {

      if (k > n)
         k = k % n;

      int[] ans = new int[n];

      for (int i = 0; i < k; i++) {
         ans[i] = nums[n - k + i];
      }

      int index = 0;
      for (int i = k; i < n; i++) {
         ans[i] = nums[index++];
      }
      return ans;
   }

   public static void main(String[] args) {
      int Array[] = { 1, 2, 3, 4, 5 };
      int N = 5;
      int K = 2;

      int[] ans = rotate(Array, N, K);
```

```
        for (int i = 0; i < N; ++i) {
            System.out.println(ans[i]);
        }
    }
}
```

2. **Problem Description -:** Given two non-negative integers n1 and n2, where n1

For example:

Suppose n1=11 and n2=15.

There is the number 11, which has repeated digits, but 12, 13, 14 and 15 have no repeated digits. So, the output is 4.

Example1:

Input:

- 11 — Vlaue of n1
- 15 — value of n2

Output:

- 4

Example 2:

Input:

- 101 — value of n1
- 200 — value of n2

Output:

- 72

<u>JAVA</u>

```
import java.util.*;
public class Main
{
    static int find(int n1, int n2) {
        int count = 0;
        for (int i = n1 ; i <= n2 ; i++) {
            int num = i;

            boolean[] visited = new boolean[10];

            while (num > 0) {
                if (visited[num % 10] == true)
                    break;
                visited[num % 10] = true;
                num /= 10;
            }
```

```
            if (num == 0)
                count++;
        }
        return count;
    }
    public static void main(String[] args) {
        int n1 = 101, n2 = 200;
        System.out.println(find(n1, n2));
    }
}
```

3. **Problem Description -:** In this 3 Palindrome, Given an input string word, split the string into exactly 3 palindromic substrings. Working from left to right, choose the smallest split for the first substring that still allows the remaining word to be split into 2 palindromes.
Similarly, choose the smallest second palindromic substring that leaves a third palindromic substring. If there is no way to split the word into exactly three palindromic substrings, print "Impossible" (without quotes). Every character of the string needs to be consumed.

Cases not allowed –

- After finding 3 palindromes using above instructions, if any character of the original string remains unconsumed.
- No character may be shared in forming 3 palindromes.

Constraints

- 1 <= the length of input sting <= 1000

Input

- First line contains the input string consisting of characters between [a-z].

Output

- Print 3 substrings one on each line.

Time Limit

1

Examples

Example 1

Input

nayannamantenet

Output

nayan
naman
tenet

Explanation

- The original string can be split into 3 palindromes as mentioned in the output.
- However, if the input was nayanamantenet, then the answer would be "Impossible".

Example 2

Input

aaaaa

Output

a
a
aaa

Explanation

- The other ways to split the given string into 3 palindromes are as follows –
- [a, aaa, a], [aaa, a, a], [aa, aa, a], etc.
- Since we want to minimize the length of the first palindromic substring using left to right processing, the correct way to split is [a, a, aaa].

Example 3

Input

aaaabaaaa

Output

a
aaabaaa
a

Explanation

- The other ways to split the given string into 3 palindromes are as follows –
- [aaaa, b, aaaa], [aa, aabaa, aa], etc.
- Since we want to minimize the length of the first palindromic substring using left to right processing, the correct way to split is [a, aaabaaa, a].

<u>Python</u>

```
import sys
def if_palindrome(s):
    if len(s)==1:
        return True
    s1=s[::-1]
    return s1==s
s=input()
l=len(s)
for i in range(1,l-1):
    s1=s[:i]
    if if_palindrome(s1):
        for j in range(1,l-i):
            s2=s[i:i+j]
            s3=s[i+j:]
            if if_palindrome(s2) and if_palindrome(s3):
                print(s1)
                print(s2)
```

```
        print(s3)
        sys.exit()
print("Impossible")
```

4. **Problem Statement -:** In this even odd problem Given a range [low, high] (both inclusive), select K numbers from the range (a number can be chosen multiple times) such that sum of those K numbers is even.
Calculate the number of all such permutations.
As this number can be large, print it modulo (1e9 +7).

Constraints

- 0 <= low <= high <= 10^9
- K <= 10^6.

Input

- First line contains two space separated integers denoting low and high respectively
- Second line contains a single integer K.

Output

- Print a single integer denoting the number of all such permutations

Time Limit

1

Examples

Example 1

Input

4 5
3

Output

4

Explanation

There are 4 valid permutations viz. {4, 4, 4}, {4, 5, 5}, {5, 4, 5} and {5, 5, 4} which sum up to an even number.

Example 2

Input

1 10
2

Output

50

Explanation

There are 50 valid permutations viz. {1,1}, {1, 3},.. {1, 9} {2,2}, {2, 4},… {2, 10} . . . {10, 2}, {10, 4},… {10, 10}. These 50 permutations, each sum up to an even number.

<u>PYTHON</u>

```
mod=1000000007
def esum(m,n,K,N):
   if(K==1):
     return n
   else:
      return (N-(m-n)*esum(m,n,K-1,N)%mod)

low,high=map(int,input().split())
K=int(input())
diff=high-low+1
if(diff%2==0):
   m=diff//2
   n=m
else:
   if(low%2==0):
      m=(diff-1)//2
      n=m+1
   else:
      m=(diff+1)//2
      n=m-1
N=m
for i in range(K-1):
   N=(N*(m+n))%mod
out=esum(m,n,K,N)%mod
print(out)
```

5. Roco is an island near Africa which is very prone to forest fire. Forest fire is such that it destroys the complete forest. Not a single tree is left.This island has been cursed by God , and the curse is that whenever a tree catches fire, it passes the fire to all its adjacent tree in all 8 directions,North, South, East, West, North-East, North-West, South-East, and South-West.And it is given that the fire is spreading every minute in the given manner, i.e every tree is passing fire to its adjacent tree.Suppose that the forest layout is as follows where T denotes tree and W denotes water.
Your task is that given the location of the first tree that catches fire, determine how long would it take for the entire forest to be on fire. You may assume that the lay out of the forest is such that the whole forest will catch fire for sure and that there will be at least one tree in the forest

Input Format:

- First line contains two integers, M, N, space separated, giving the size of the forest in terms of the number of rows and columns respectively.
- The next line contains two integers X,Y, space separated, giving the coordinates of the first tree that catches the fire.
- The next M lines, where ith line containing N characters each of which is either T or W, giving the position of the Tree and Water in the ith row of the forest.

Output Format:

Single integer indicating the number of minutes taken for the entire forest to catch fire

Constrains:

- $3 \le M \le 20$
- $3 \le N \le 20$

Sample Input 1:

3 3
W T T
T W W
W T T
Sample Output 1:
5

Explanation:
In the second minute,tree at (1,2) catches fire,in the third minute,the tree at (2,1) catches fire,fourth minute tree at (3,2) catches fire and in the fifth minute the last tree at (3,3) catches fire.
Sample Input 2:
6 6
1 6
W T T T T T
T W W W W W
W T T T T T
W W W W W T
T T T T T T
T W W W W W

Sample Output 2:
16

<u>Python</u>

```
ar=[]
n=0
br=[(-1,0),(+1,0),(0,+1),(0,-1),(-1,+1),(-1,-1),(+1,+1),(+1,-1)]
def bfs(i,j):
    key=(i,j)
    vis=set()
    vis.add(key)
    dis=dict()
    dis[key]=1
    que=[]
    que.append((i,j))
    time=1
    while(que):
        x,y=que.pop(0)
        for dx,dy in br:
            nx,ny=x+dx , y+dy
```

```
        key=(nx,ny)
        if(0<=nx<=n-1 and 0<=ny<=n-1 and key not in vis and ar[nx][ny]=='T'):
            vis.add(key)
            dis[key]=dis[(x,y)]+1
            que.append((nx,ny))
            time=max(time,dis[key])
    return time

n,m=map(int,input().split())
r,c=map(int,input().split())
ar=[]
for i in range(n):
    ar.append(tuple(map(str,input().split())))

print(bfs(r-1,c-1))
```

6. **Problem Statement:-**

Compute the nearest larger number by interchanging its digits updated.Given 2 numbers a and b find the smallest number greater than b by interchanging the digits of a and if not possible print -1.

- **Input Format**
  2 numbers a and b, separated by space.
- **Output Format**
  A single number greater than b.

If not possible, print -1

- **Constraints**
  1 <= a,b <= 10000000

Example 1:
Sample Input:
459 500
Sample Output:
549

Example 2:
Sample Input:
645757 457765
Sample Output:
465577

Python

```
import sys
a=input().strip()
b=input().strip()
a=''.join(sorted(a)[::-1])
if int(a)<=int(b):
```

```
    print(-1)
    sys.exit()
else:
    if len(a)>len(b):
        print(int(a[::-1]))
        sys.exit()
    else:
        n=len(b)
        a=list(a[::-1])
        s=''
        n1=len(a)
        for i in range(n):
            d=b[i]
            j=0
            while a[j]<b[i] and n1:
                j+=1
            if a[j]>b[i]:
                s+=a[j]
                a.remove(a[j])
                a.sort()
                s+=''.join(a)
                print(int(s))
                sys.exit()
            else:
                s1=s+a[j]
                g=a.copy()
                g.remove(a[j])
                if n1>1:
                    g.sort()
                    g=g[::-1]
                    s1+=''.join(g)
                if int(s1)>int(b):
                    s+=a[j]
                    a.remove(a[j])
                    n1-=1
                else:
                    while a[j]<=b[i]:
                        j+=1
                    s+=a[j]
                    a.remove(a[j])
                    a.sort()
                    s+=''.join(a)
                    print(int(s))
                    sys.exit()
print(int(s))
```

7. **Problem Statement:-** In a Conference ,attendees are invited for a dinner after the conference.The Co-ordinator,Sagar arranged around round tables for dinner and want to have an impactful seating experience for the attendees.Before finalizing the seating arrangement,he wants to analyze all the possible arrangements.These are R round tables and N attendees.In case where N is an exact multiple of R,the number of attendees must be exactly N//R,,If N is not an exact multiple of R, then the distribution of attendees must be as equal as possible.Please refer to the example section before for better understanding.
For example, R = 2 and N = 3
All possible seating arrangements are
(1,2) & (3)
(1,3) & (2)
(2,3) & (1)
Attendees are numbered from 1 to N.

Input Format:

- The first line contains T denoting the number of test cases.
- Each test case contains two space separated integers R and N, Where R denotes the number of round tables and N denotes the number of attendees.

Output Format:
Single Integer S denoting the number of possible unique arrangements.

Constraints:

- 0 <= R <= 10(Integer)
- 0 < N <= 20 (Integer)

Sample Input 1:
1
2 5

Sample Output 1:
10

Explanation:
R = 2, N = 5
(1,2,3) & (4,5)
(1,2,4) & (3,5)
(1,2,5) & (3,4)
(1,3,4) & (2,5)
(1,3,5) & (2,4)
(1,4,5) & (2,3)
(2,3,4) & (1,5)
(2,3,5) & (1,4)
(2,4,5) & (1,3)
(3,4,5) & (1,2)
Arrangements like
(1,2,3) & (4,5)

(2,1,3) & (4,5)
(2,3,1) & (4,5) etc.
But as it is a round table,all the above arrangements are same.

Python

```
testcases = int(input())
for i in range(testcases):
    tables, people = map(int, input().split())
    result = 1

    if tables >= people:
        print(result)
    else:
        PA = people // tables
        PB = PA + 1
        TB = people % tables
        TA = tables - TB

        # Using DP to store factorials pre-hand
        fact = [1]
        for j in range(1, people + 1):
            fact.append(j*fact[j-1])

        for i in range(TB):
            result = result * (fact[people]//(fact[PB]*fact[people-PB]))
            people = people - PB

        for i in range(TA):
            result = result * (fact[people]//(fact[PA]*fact[people-PA]))
            people = people - PA

        print(result)
```

8. Problem

1, 2, 1, 3, 2, 5, 3, 7, 5, 11, 8, 13, 13, 17, ……..

This series is a mixture of 2 series – all the odd terms in this series form a Fibonacci series and all the even terms are the prime numbers in ascending order.

Write a program to find the Nth term in this series.

- The value N is a Positive integer that should be read from STDIN.
- The Nth term that is calculated by the program should be written to STDOUT.
- Other than the value of Nth term, no other characters/strings or message should be written to STDOUT.

For example, when N = 14, the 14th term in the series is 17. So only the value 17 should be printed to STDOUT.

```python
def fibbo(n):
    n1, n2 = 0, 1
    count = 0
    if n <= 0:
       print("Please enter a positive integer")
    elif n == 1:
       print(n1)
    else:
       while count < n:
           nth = n1 + n2
           n1 = n2
           n2 = nth
           count += 1
    print(n1)

def prime(n):
    count = 0
    for i in range(2, 99999):
        flag=0
        for j in range(2,i):
            if (i % j == 0):
                flag=1
                break
        if flag == 0:
            count+=1
            if count == n:
                print(i)
                break

# main
n = int(input())
if n%2 == 1:
    fibbo(n//2+1)
else:
    prime(n//2)
```

9. Selection sort is a simple sorting algorithm. This sorting algorithm is an in-place comparison-based algorithm in which the list is divided into two parts, the sorted part at the left end and the unsorted part at the right end. Initially, the sorted part is empty and the unsorted part is the entire list.

The smallest element is selected from the unsorted array and swapped with the leftmost element, and that element becomes a part of the sorted array. This process continues moving unsorted array boundary by one element to the right.

This algorithm is not suitable for large data sets as its average and worst case complexities are of $O(n^2)$, where **n** is the number of items.'

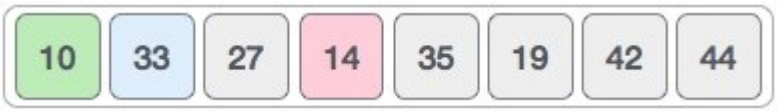
Consider the following depicted array as an example.

For the first position in the sorted list, the whole list is scanned sequentially. The first position where 14 is stored presently, we search the whole list and find that 10 is the lowest value.

10 is placed at 0th index of array, and we increase the value of loop by +1.

Starting from 1st position of array, we selected 33 as minimum element initially.

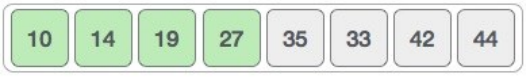
Then we found out that 14 is 2nd lowest element after searching the whole array starting from 1st position of length-1 position

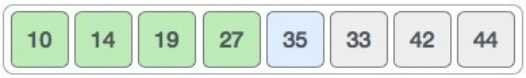


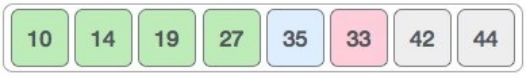
After two iterations, two least values are positioned at the beginning in a sorted manner.

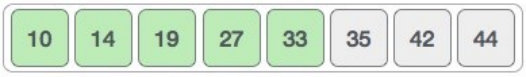
The same process is applied to the rest of the items in the array.

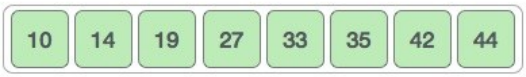
Following is a pictorial depiction of the entire sorting process –











```
# Python2 program for implementation of Selection
# Sort
import sys
A = [64, 25, 12, 22, 11]

# Traverse through all array elements
for i in range(len(A)):

# Find the minimum element in remaining
# unsorted array
    min_idx = i
    for j in range(i+1, len(A)):
        if A[min_idx] > A[j]:
            min_idx = j

# Swap the found minimum element with
# the first element
    A[i], A[min_idx] = A[min_idx], A[i]

# Driver code to test above
print (“Sorted array”)
for i in range(len(A)):
    print(“%d“ %A[i]),
```

10. Count number of occurrences (or frequency) in a sorted array

Given a sorted array arr[] and a number x, write a function that counts the occurrences of x in arr[]. The expected time complexity is O(Logn)

Examples:

Input: arr[] = {1, 1, 2, 2, 2, 2, 3,}, x = 2
Output: 4 // x (or 2) occurs 4 times in arr[]

Input: arr[] = {1, 1, 2, 2, 2, 2, 3,}, x = 3
Output: 1

Input: arr[] = {1, 1, 2, 2, 2, 2, 3,}, x = 1
Output: 2

Input: arr[] = {1, 1, 2, 2, 2, 2, 3,}, x = 4
Output: -1 // 4 doesn't occur in arr[]

```
# Python3 program to count
# occurrences of an element

# Returns number of times x
# occurs in arr[0..n-1]
```

```
def countOccurrences(arr, n, x):
    res = 0
    for i in range(n):
        if x == arr[i]:
            res += 1
    return res

# Driver code
arr = [1, 2, 2, 2, 2, 3, 4, 7 ,8 ,8]
n = len(arr)
x = 2
print (countOccurrences(arr, n, x))
```

11. Longest Common Subsequence –

Let us discuss Longest Common Subsequence (LCS) problem as one more example problem that can be solved using Dynamic Programming.

LCS Problem Statement: Given two sequences, find the length of longest subsequence present in both of them. A subsequence is a sequence that appears in the same relative order, but not necessarily contiguous. For example, “abc”, “abg”, “bdf”, “aeg”, ‘”acefg”, .. etc are subsequences of “abcdefg”. So a string of length n has 2^n different possible subsequences.

It is a classic computer science problem, the basis of diff(a file comparison program that outputs the differences between two files), and has applications in bioinformatics.

Examples:

LCS for input Sequences “ABCDGH” and “AEDFHR” is “ADH” of length 3.

LCS for input Sequences “AGGTAB” and “GXTXAYB” is “GTAB” of length 4.

```
# A Naive recursive Python implementation of LCS problem
def lcs(X, Y, m, n):
    if m == 0 or n == 0:
        return 0;
    elif X[m-1] == Y[n-1]:
        return 1 + lcs(X, Y, m-1, n-1);
    else:
        return max(lcs(X, Y, m, n-1), lcs(X, Y, m-1, n));

# Driver program to test the above function
X = “AGGTAB”
Y = “GXTXAYB”
print “Length of LCS is “, lcs(X , Y, len(X), len(Y))
```

12. SubString Problem in C

Given a string as an input. We need to write a program that will print all non-empty substrings of that given string.

Python

```
def subString(Str,n):
   # take starting point
   for Len in range(1,n + 1):

      # Pick ending point
      for i in range(n – Len + 1):

         # Print characters from current
         # starting point to ending point
         j = i + Len – 1

         for k in range(i,j + 1):
            print(Str[k],end="")
         print()

# Driver program to test above function
str1 = "muskan"
subString(str1,len(str1))
```

13. Pythrogorous Triplets

A Pythagorean triplet is a set of three integers a, b and c such that $a^2 + b^2 = c^2$. Given a limit, generate all Pythagorean Triples with values smaller than given limit.

Input : limit = 20

Output : 3 4 5
8 6 10
5 12 13
15 8 17
12 16 20

A **Simple Solution** is to generate these triplets smaller than given limit using three nested loop. For every triplet, check if Pythagorean condition is true, if true, then print the triplet. Time complexity of this solution is $O(limit^3)$ where 'limit' is given limit.

An **Efficient Solution** can print all triplets in O(k) time where k is number of triplets printed. The idea is to use square sum relation of Pythagorean triplet, i.e., addition of squares of a and b is equal to square of c, we can write these number in terms of m and n such that,

$a = m^2 - n^2$
$b = 2 * m * n$
$c = m^2 + n^2$

because,

$a^2 = m^4 + n^4 – 2 * m^2 * n^2$
$b^2 = 4 * m^2 * n^2$
$c^2 = m^4 + n^4 + 2* m^2 * n^2$

We can see that $a^2 + b^2 = c^2$, so instead of iterating for a, b and c we can iterate for m and n and can generate these triplets.
Below is the implementation of above idea :

Python

```
# triplets smaller than a given limit
# Function to generate pythagorean
# triplets smaller than limit
def pythagoreanTriplets(limits) :
    c, m = 0, 2

    # Limiting c would limit
    # all a, b and c
    while c < limits :

    # Now loop on n from 1 to m-1
        for n in range(1, m) :
            a = m * m – n * n
            b = 2 * m * n
            c = m * m + n * n

        # if c is greater than
            # limit then break it
            if c > limits :
                break
            print(a, b, c)
        m = m + 1

# Driver Code
if __name__ == '__main__' :

    limit = 20
    pythagoreanTriplets(limit)
```

14. Armstrong Number

Given a number x, determine whether the given number is Armstrong number or not. A positive integer of **n digits** is called an Armstrong number of **order n** (order is number of digits) if:
abcd... = pow(a,n) + pow(b,n) + pow(c,n) + pow(d,n) + ...

Example:
Input : 153
Output : Yes
153 is an Armstrong number.

1*1*1 + 5*5*5 + 3*3*3 = 153

Input : 120
Output : No
120 is not a Armstrong number.
1*1*1 + 2*2*2 + 0*0*0 = 9

Input : 1253
Output : No
1253 is not a Armstrong Number
1*1*1*1 + 2*2*2*2 + 5*5*5*5 + 3*3*3*3 = 723

Input : 1634
Output : Yes
1*1*1*1 + 6*6*6*6 + 3*3*3*3 + 4*4*4*4 = 1634

Python

```
# Python program to determine whether the number is
# Armstrong number or not
# Function to calculate x raised to the power y
def power(x, y):
    if y==0:
        return 1
    if y%2==0:
        return power(x, y//2)*power(x, y//2)

    return x*power(x, y//2)*power(x, y//2)

# Function to calculate order of the number
def order(x):

# variable to store of the number
    n = 0
    while (x!=0):
        n = n+1
        x = x//10
    return n

# Function to check whether the given number is
# Armstrong number or not
def isArmstrong (x):
    n = order(x)
    temp = x
    sum1 = 0
    while (temp!=0):
```

```
        r = temp%10
        sum1 = sum1 + power(r, n)
        temp = temp//10

    # If condition satisfies
    return (sum1 == x)

# Driver Program
x = 153
print(isArmstrong(x))
x = 1253
print(isArmstrong(x))
```

15. Counting frequencies of array elements

Given an array which may contain duplicates, print all elements and their frequencies.

Input : arr[] = {10, 20, 20, 10, 10, 20, 5, 20}

Output : 10 3

20 4

5 1

Input : arr[] = {10, 20, 20}

Output : 10 1

20 2

<u>Python</u>

```
# Input size of array
n = int(input("Enter the number of elements : "))

# Input elements in array
print("Enter elements in space seperated: ")
arr=list(map(int,input().split()))
s=set(arr)

print("Frequency of all elements of array : n")
for i in s:
    print(i, " occurs ", arr.count(i), " times ")
```

16. Basic Program for Decimal to Octal

Given a decimal number as input, we need to write a program to convert the given decimal number into equivalent octal number. i.e convert the number with base value 10 to base value 8. The base

value of a number system determines the number of digits used to represent a numeric value. For example, the binary number system uses two digits 0 and 1, octal number system uses 8 digits from 0-7 and decimal number system uses 10 digits 0-9 to represent any numeric value.

Examples:
Input : 16
Output : 20

Input : 10
Output : 12

Input: 33
Output: 41

Algorithm:

1. Store the remainder when the number is divided by 8 in an array.
2. Divide the number by 8 now
3. Repeat the above two steps until the number is not equal to 0.
4. Print the array in reverse order now.

For Example:
If the given decimal number is 16.
Step 1: Remainder when 16 is divided by 8 is 0. Therefore, arr[0] = 0.
Step 2: Divide 16 by 8. New number is 16/8 = 2.
Step 3: Remainder when 2 is divided by 8 is 2. Therefore, arr[1] = 2.
Step 4: Divide 2 by 8. New number is 2/8 = 0.
Step 5: Since number becomes = 0. Stop repeating steps and print the array in reverse order.
Therefore the equivalent octal number is 20.

Python

```
def DecimalToOctal(decimal):
   octal = 0
   i = 1
   while (decimal != 0):
      octal = octal + (decimal % 8) * i
      decimal = int(decimal / 8)
      i = i * 10
   return octal

print(DecimalToOctal(int(input())))
```

17. Basic Program for Binary to Octal Conversion
Given a binary number as input, we need to write a program to convert the given binary number into equivalent octal number. i.e convert the number with base value 2 to base value 8. The base value of a number system determines the number of digits used to represent a numeric value. For

example, the binary number system uses two digits 0 and 1, octal number system uses 8 digits from 0-7 and decimal number system uses 10 digits 0-9 to represent any numeric value.

Python

```
print("Octal:",str(oct(int(input("Enter a binary number:"),2)))[2:])
```

Output:

```
Enter a binary number:111
Octal:7
```

18. GCD of Three Numbers

Definition of HCF (Highest common factor):

HFC is also called the greatest common divisor (gcd). HCF of two numbers is the largest positive number which can divide both numbers without any remainder. For example, the HCF of two numbers 4 and 8 is 2 since 2 is the largest positive number which can divide 4 as well as 8 without a remainder.

The logic for writing program:

It is clear that any number is not divisible by greater than the number itself.

☆In the case of more than one number, a possible maximum number that can divide all of the numbers must be a minimum of all of that numbers.

For example 10, 20, and 30

Min (10, 20, 30) =10 can divide all there numbers. So we will take one for loop which will start from the min of the numbers and will stop the loop when it became one since all numbers are divisible by one. Inside for loop, we will write one if conditions which will check divisibility of both the numbers.

Python

```
def findGCD(x, y):
   while(y):
      x, y = y, x % y
   return x

l = [2, 4, 6]

num1=l[0]
num2=l[1]
gcd=findGCD(num1,num2)

for i in range(2,len(l)):
   gcd=findGCD(gcd,l[i])

print(gcd)
```

19. **Pushing all 0's**

Given an array of random numbers, Push all the zero's of a given array to the end of the array. For example, if the given arrays is {1, 9, 8, 4, 0, 0, 2, 7, 0, 6, 0}, it should be changed to {1, 9, 8, 4, 2, 7, 6, 0, 0, 0, 0}. The order of all other elements should be same. Expected time complexity is O(n) and extra space is O(1).

Example:

Input : arr[] = {1, 2, 0, 4, 3, 0, 5, 0};
Output : arr[] = {1, 2, 4, 3, 5, 0, 0};

Input : arr[] = {1, 2, 0, 0, 0, 3, 6};
Output : arr[] = {1, 2, 3, 6, 0, 0, 0};

```python
# Python3 code to move all zeroes
# at the end of array

# Function which pushes all
# zeros to end of an array.
def pushZerosToEnd(arr, n):
    # Count of non-zero elements
    count = 0

    # Traverse the array. If element encountered is non-zero, then
    # replace the element at index 'count' with this element
    for i in range(n):
        if arr[i] != 0:
            # count is incremented
            arr[count] = arr[i]
            count+=1

    # Now all non-zero elements have been shifted to front and 'count' is set
    # as index of first 0. Make all elements 0 from count to end.
    while count < n:
        arr[count] = 0
        count += 1

# Driver code
arr = [1, 6, 7, 4, 0, 3, 0, 2, 7, 0, 6, 0, 9]
n = len(arr)
pushZerosToEnd(arr, n)
print("Array after pushing all zeros to end of array:")
print(arr)
```

20. **You are choreographic a circus shows with various animals. For one act, you are given two kangaroos on a number line ready to jump in the positive (i.e towards positive infinity).**

- The first kangaroos start at location x1 and move at a rate of v1 meter per jump.
- The second kangaroos start at location x2 and move at a rate of v2 meters per jump.

You have to figure out a way to get both kangaroos at the same location at the same time as part of the shows. If it is possible, return YES, otherwise return NO.

For example: – kangaroos 1 starts at x1=2 with a jump distance v1=1 and kangaroos 2 starts at x2=1 with a jump distance of v2. After on jump, they are both at x=3. (x1+v1=2 x2+v2=1+2), So our answer is YES.

Python

```
x1,v1,x2,v2 =map(int,input().split())
x= [x1,v1]
y= [x2,v2]
back= min(x,y)
fwd= max(x,y)
dist= fwd[0]-back[0]

while back[0] <fwd[0]:
    back[0] += back[1]
    fwd[0] += fwd[1]
    if fwd[0]-back[0]>= dist :
        break
print(["NO" , "YES"] [back[0] == fwd[0]])
```

21. You will be given two arrays of integers and asked to determine all integer that satisfies the following two conditions:

1. The elements of the first array are all factors of the integer being considered
2. The integer being considered is a factor of all elements of the second array.

These numbers are referred to as being between the two arrays. You must determine how much such a number exists.

Python

```
import sys

n,m=map(int,input().split())
a= map(int,input().split())
b= map(int,input().split())
cnt  = 0
for i in range(1, 101 ):
    works = True
    for j in a:
        if i % j != 0:
            works = false
    for j in b:
        if j % i != 0:
            works  = False
    if works :
```

```
        cnt += 1
print (cnt)
```

22. **You have been asked to help study the population of birds migrating across the continent. Each type of bird you are interested in will be identified by an integer value. Each time a particular kind of bird is spotted, its id number will be added to your array of sightings.**

    You would like to be able to find out which type of bird is most common given a list of sightings. Your task is to print the type number of that bird and if two or more types of birds are equally common, choose the type with the smallest ID number.

    For example, assume your bird sightings are of types arr = [1,1,2,2,3]. There are two each of types 1 and 2, and one sighting of type 3. Pick the lower of the two types seen twice: type 1.

    Python

```
n=int(input())
arr=list(map(int,input().split()))
out=[]
for i in range(1,6):
    out.append(arr.count(i))
print(out.index(max(out))+1)
```

23. **A discrete Mathematics professor has a class of students. Frustrated with their lack of discipline, decides to cancel class if fewer than some number of students are present when class starts. Arrivel times go from on time (arrival time<=0) to arrived late (arrivalTime> 0).**
    Given the arrival time of each student and a threshold number of attendees, determine if the class is canceled.

    Python

```
t = int(input())
for T in range(t):
    n,k = map(int,input().split())
    a = list(map(int, input().split()))
    count=0
    for i in range(n):
        if a[i]<=0:
            count+=1
    if count< k:
        print("YES")
    else:
        print("NO")
```

```
Output:

4 3
-1 -3 4 2
```

```
YES
4 2
0 -1 2 1
NO
```

24. **Anna and Brian are sharing a meal at a restaurant and they agree to split the bill equally. Brian wants to order something that Anna is allergic to though, and they agree that Anna won't pay for that item. Brian gets the check and calculates Anna's portion. You must determine if his calculation is correct.**
For example, assume the bill has the following prices:bill=[2,4,6]. Anna declines to eat item k=bill[2] which costs 6. If Brian calculates the bill correctly, Anna will pay (2+4)/2=3. If he includes the cost of bill[2],he will calculate (2+4+6)/2 = 6. In the second case, he should refund 3 to Anna.

Python

```
n,k = map(int,input().split())
bill = list(map(int, input().split()))
b = int(input())
s=sum(bill)-bill[k]
if (s//2)<b:
    print(b-(s//2))
else:
    print("Bon Appetit")
```

25. **The Utopian Tree goes through *2* cycles of growth every year. Each spring, it *doubles* in height. Each summer, its height increases by *1* meter. Laura plants a Utopian Tree sapling with a height of *1* meter at the onset of spring. How tall will her tree be after growth cycles?**
For example, if the number of growth cycles is n=5, the calculations are as follows:
Periods Height
    - 1
    - 2
    - 3
    - 6
    - 7
    - 14

Python

```
for i in range(int(input())):
    n=int(input())
    h=1
    for i in range(1,n+1):
            if i&1:
                h*=2
            else:
                h+=1
```

```
    print(h)
```

26. **John Watson knows of an operation called a *right circular rotation* on an array of integers. One rotation operation moves the last array element to the first position and shifts all remaining elements right one. To test Sherlock's abilities, Watson provides Sherlock with an array of integers. Sherlock is to perform the rotation operation a number of times then determine the value of the element at a given position.**
For each array, perform a number of right circular rotations and return the value of the element at a given index.

For example, array a=[3,4,5], number of rotation, k=2 and indices to check, .
m=[1,2] First we perform the two rotations:
[3,4,5]à [5,3,4]à [4,5,3]

Now return the values from the zero-based indices 1 and 2 as indicated in the m array.
a[1]=5
a[2]=3

Python

```
n,k,q=map(int,input().split())
arr=list(map(int,input().split()))
for i in range(n-k%n):
    a=arr.pop(0)
    arr.append(a)
for i in range(q):
    print(arr[int(input())])
```

27. **An integer d is a *divisor* of an integer n if the remainder of n/d = 0.**
Given an integer, for each digit that makes up the integer determine whether it is a divisor. Count the number of divisors occurring within the integer.

Note: Each digit is considered to be unique, so each occurrence of the same digit should be counted (e.g. for n=111,1is a divisor of 111 each time it occurs so the answer is 3).

Python

```
t = int(input())
for t_itr in range(t):
    n = int(input())
    s=str(n)
    count=0
    for i in s:
        if int(i)!=0 and n%int(i)==0:
            count+=1
    print(count)
```

28. **We define the distance between two array values as the number of indices between the two values.**
Given a, find the minimum distance between any pair of equal elements in the array. If no such value exists, print-1.

Python

```
import sys
n=int(input())
A=list(map(int,input().split()))
for d in range(1, n):
    for i in range(n - d):
        if A[i] == A[i+d]:
            print(d)
            sys.exit(0)
print(-1)
```

29. **Monica wants to buy a keyboard and a USB drive from her favorite electronics store. The store has several models of each. Monica wants to spend as much as possible for 2 the items, given her budget.**
Given the price lists for the store's keyboards and USB drives, and Monica's budget, find and print the amount of money Monica will spend. If she doesn't have enough money to both a keyboard *and* a USB drive, print -1 instead. She will buy only the two required items.

Python

```
b,n,m = map(int,input().split())
keyboards=list(map(int,input().split()))
drives=list(map(int,input().split()))
out = []
for x in keyboards:
    for y in drives:
        if x+y <= b:
            out.append(x+y)
if not out:
    print(-1)
else:
    print(max(out))
```

30. **Larry has been given a permutation of a sequence of natural numbers incrementing 1 from an array. He must determine whether the array can be sorted using the following operation any number of times:**

- Choose any 3 consecutive indices and rotate their elements in such a way thatABC-> BCA->CAB->ABC.

Python

```
def larrysArray(A):
    inversion = 0
    for i in range(len(A)):
        for j in range(i+1, len(A)):
            if (A[i] > A[j]):
                inversion += 1

    if inversion % 2 == 0:
        return 'YES'
    else:
        return 'NO'

print(larrysArray([3,1,2]))
print(larrysArray([2,1,3]))
```

31. **Consider an array of numeric strings where each string is a positive number with anywhere from 1to 10^6 digits. Sort the array's elements in *non-decreasing*, or ascending order of their integer values and print each element of the sorted array on a new line.**

Python

```
import sys

n = int(input().strip())
unsorted = []
unsorted_i = 0
for unsorted_i in range(n):
    unsorted_t = str(input().strip())
    unsorted.append(unsorted_t)
    ans=sorted(unsorted, key=lambda x: (len(x),x))
for s in ans:
    print(s)
```

32. **Write a program to check whether a given number is a happy number or an unhappy number.**

Happy number: Starting with any positive integer, replace the number by the sum of the squares of its digits, and repeat the process until the number equals 1, or it loops endlessly in a cycle which does not include 1.

Python

```
# Python3 program to check if a number
# is happy number or not
```

```python
# Returns sum of squares of digits
# of a number n. For example for n = 12
# it returns 1 + 4 = 5
def sumDigitSquare( n):
    sumsq = 0;
    while (n!=0):
        digit = n % 10
        sumsq += digit * digit
        n = n // 10

    return sumsq;


# Returns true if n is Happy number
# else returns false.
def isHappy(n):
    # A set to store numbers during
    # repeated square sum process
    s=set()
    s.add(n)

    # Keep replacing n with sum of squares of digits
    # until we either reach 1 or we endup in a cycle
    while (True):

        # Number is Happy if we reach 1
        if (n == 1):
            return True;

        # Replace n with sum of squares
        # of digits
        n = sumDigitSquare(n)

        # If n is already visited, a cycle
        # is formed, means not Happy
        if n in s:
            return False

        # Mark n as visited
        s.add(n)

    return false;


# Driver code
n = 4
```

```
if (isHappy(n)):
    print("Yes")
else:
    print("No")
```

33. **John works at a clothing store. He has a large pile of socks that he must pair by color for sale. Given an array of integers representing the color of each sock, determine how many pairs of socks with matching colors there are.**

Function Description:
Complete the sockMerchant() function in the editor below. It must return an integer representing the number of matching pairs of socks that are available.
sockMerchant has the following parameter(s):
n: the number of socks in the pile
ar: the colors of each sock.

Input Format
The first line contains an integer, the number of socks represented in.
The second line contains space-separated integers describing the colors of the socks in the pile.

Output Format
Return the total number of matching pairs of socks that John can sell.

Sample Input
9
10 20 20 10 10 30 50 10 20

Sample Output
3

<u>Python</u>

```
# Python3 program to calculate matching pair of socks

# function which will return the number of atching pairs
def matchingSocks(arr,n):
    # set of disctinct type of socks
    s=set(arr)
    #count variable
    c=0
    for i in s:
        c+=arr.count(i)//2
    return c

#Driver Program
n=9
arr=[10 ,20, 20, 10, 10, 30, 50, 10, 20]
result=matchingSocks(arr,n)
```

```
print(result)
```

34. Gary is an avid hiker. He tracks his hikes meticulously, paying close attention to small details like topography. During his last hike he took exactly **n** steps. For every step he took, he noted if it was an *uphill* **U**, or a *downhill*, **D** step. Gary's hikes start and end at sea level and each step up or down represents a 1 unit change in altitude. We define the following terms:

- A *mountain*is a sequence of consecutive steps *above* sea level, starting with a step *up* from sea level and ending with a step *down* to sea level.
- A *valley*is a sequence of consecutive steps *below* sea level, starting with a step *down* from sea level and ending with a step *up* to sea level.

Given Gary's sequence of *up* and *down* steps during his last hike, find and print the number of *valleys* he walked through.

For example, if Gary's path is **s=[DDUUUUDD]** he first enters a valley 2 units deep. Then he climbs out an up onto a mountain 2 units high. Finally, he returns to sea level and ends his hike.

Function Description

Complete the *countingValleys* function in the editor below. It must return an integer that denotes the number of valleys Gary traversed.

countingValleys has the following parameter(s):

- *n*: the number of steps Gary takes
- *s*: a string describing his path

Input Format

The first line contains an integer n , the number of steps in Gary's hike.

The second line contains a single string **s** of **n** characters that describe his path.

Print a single integer that denotes the number of valleys Gary walked through during his hike.

Sample Input

8

UDDDUDUU

Sample Output

1

<u>Python</u>

```
def noOfValley(n,s):
    sealevel=0
    valley=0
    for i in s:
        if i=='U':
            sealevel+=1
        else:
            if sealevel==0:
                valley+=1
                sealevel-=1
```

```
    return valley

n=int(input())
s=input()
result=noOfValley(n,s)
print(result)
```

35. You are in charge of the cake for your niece's birthday and have decided the cake will have one candle for each year of her total age. When she blows out the candles, she'll only be able to blow out the tallest ones. Your task is to find out how many candles she can successfully blow out.
For example, if your niece is turning **4** years old, and the cake will have **4** candles of height **4 ,1 ,1 ,3** she will be able to blow out **2** candles successfully, since the tallest candles are of height **4** and there are **2** such candles.

Function Description
Complete the function birthdayCakeCandles in the editor below. It must return an integer representing the number of candles she can blow out.
birthdayCakeCandles has the following parameter(s):

- *ar*: an array of integers representing candle heights

Input Format
The first line contains a single integer **n** denoting the number of candles on the cake.
The second line contains **n** space-separated integers, where each integer **i** describes the height of candle **i** .

Output Format
Return the number of candles that can be blown out on a new line.

Sample Input 0
4
3 2 1 3

Sample Output 0
2

<u>Python</u>

```
n = input()
arr = list(map(int,input().split(" ")))
print(arr.count(max(arr)))
```

36. Monica wants to buy a keyboard and a USB drive from her favorite electronics store. The store has several models of each. Monica wants to spend as much as possible for the **2** items, given her budget. Given the price lists for the store's keyboards and USB drives, and Monica's budget, find and print the amount of money Monica will spend. If she doesn't have enough money to both a keyboard *and* a USB drive, print -1 instead. She will buy only the two required items.

For example, suppose she has **b=60** to spend. Three types of keyboards cost **keyboards=[4,50,60]**. Two USB drives cost **drives=[5,8,12]**. She could purchase a **40 keyboards + 12 USB drives =52** or a **50 keyboards + 8 USB drives =58**. She chooses the latter. She can't buy more than **2** items so she can't spend exactly **60**.

Function Description

Complete the *getMoneySpent* function in the editor below. It should return the maximum total price for the two items within Monica's budget, or **-1** if she cannot afford both items.

getMoneySpent has the following parameter(s):

- *keyboards*: an array of integers representing keyboard prices
- *drives*: an array of integers representing drive prices
- *b*: the units of currency in Monica's budget

Input Format

The first line contains three space-separated integers **b ,n**, and **m** , her budget, the number of keyboard models and the number of USB drive models.

The second line contains **n** space-separated integers **keyboard[i]**, the prices of each keyboard model.

The third line contains **m** space-separated integers **drives**, the prices of the USB drives.

Output Format

Print a single integer denoting the amount of money Monica will spend. If she doesn't have enough money to buy one keyboard *and* one USB drive, print -1 instead.

Sample Input 0

10 2 3
3 1
5 2 8

Sample Output 0

9

<u>Python</u>

```
# Python3 Program

# Function to calculate how much money spent
def getMoneySpent(keyboards, drives, b):
    maxval = 0
    for x in keyboards:
        for y in drives:
            s= x+y
            if s>maxval and s<=b:
                maxval= s
    return maxval or -1

#Driver Code
b,n,m = map(int,input().split())
k  = list(map(int,input().split()))
d  = list(map(int,input().split()))
print(getMoneySpent(k, d, b))
```

37. Two cats and a mouse are at various positions on a line. You will be given their starting positions. Your task is to determine which cat will reach the mouse first, assuming the mouse doesn't move and the cats travel at equal speed. If the cats arrive at the same time, the mouse will be allowed to move and it will escape while they fight.

You are given **q** queries in the form of **x ,y** and ,**z** representing the respective positions for cats **A** and **B** and for mouse **C** .
Complete the function **catAndMouse** to return the appropriate answer to each query, which will be printed on a new line.

- If cat **A**catches the mouse first, print Cat A.
- If cat **B**catches the mouse first, print Cat B.
- If both cats reach the mouse at the same time, print Mouse Cas the two cats fight and mouse escapes.

For example, cat **A** is at position **x=2** and cat **B** is at **y=5** . If mouse **C** is at position **z=4**,It is **2** units from cat **A** and **1** unit from cat **B**. Cat **B** will catch the mouse.

Function Description
Complete the *catAndMouse* function in the editor below. It should return one of the three strings as described.
catAndMouse has the following parameter(s):

- *x*: an integer, Cat **A**'s position
- *y*: an integer, Cat **B'**s position
- *z*: an integer, Mouse **C**'s position

Input Format
The first line contains a single integer **q**, denoting the number of queries.
Each of the **q** subsequent lines contains three space-separated integers describing the respective values of **x** (catA 's location) ,**y** (catB 's location), and **z** (mouse C's location).

Output Format
For each query, return Cat A if cat A catches the mouse first, Cat B if cat B catches the mouse first, or Mouse C if the mouse escapes.

Sample Input 0
2
1 2 3
1 3 2

Sample Output 0
Cat B
Mouse C

<u>Python</u>

```
# Python3 Program
import sys

# Number of Test Cases
q = int(input().strip())
```

```
for a in range(q):
    x,y,z = input().strip().split(‘ ‘)
    x,y,z = [int(x),int(y),int(z)]
    d1 = abs(x-z)
    d2 = abs(y-z)
    if d1 == d2:
        print(‘Mouse C’)
    elif d1 < d2:
        print(‘Cat A’)
    elif d1 > d2:
        print(‘Cat B’)
```

38. You will be given an array of integers. All of the integers except one occur twice. That one is unique in the array.

Given an array of integers, find and print the unique element.

For example **a=[1,2,3,4,3,2,1]** the unique element is **4**.

Function Description

Complete the *lonelyinteger* function in the editor below. It should return the integer which occurs only once in the input array.

lonelyinteger has the following parameter(s):

- *a*: an array of integers

Input Format

The first line contains a single integer **n** denoting the number of integers in the array.

The second line contains **n** space-separated integers describing the values in **a**.

Output Format

Print the unique integer in the array.

Sample Input 0

1

1

Sample Output 0

1

<u>Python</u>

```
Python3 program to find the unique element from the array

# function will return the unique element
def unique(arr):
    # set of distinct elements
    s=set(arr)
    for i in s:
        if arr.count(i)==1:
            return i
```

```
# Driver Program
arr=[1,2,3,4,3,2,1]
result=unique(arr)
print(result)
```

39. **2022 – September Day 1**

Problem Statement – An automobile company manufactures both a two wheeler (TW) and a four wheeler (FW). A company manager wants to make the production of both types of vehicle according to the given data below:

- 1st data, Total number of vehicle (two-wheeler + four-wheeler)=v
- 2nd data, Total number of wheels = W

The task is to find how many two-wheelers as well as four-wheelers need to manufacture as per the given data.

Example :

Input :

200 -> Value of V

540 -> Value of W

Output :

TW =130 FW=70

Explanation:

130+70 = 200 vehicles

(70*4)+(130*2)= 540 wheels

Constraints :

- 2<=W
- W%2=0
- V<W

Print "INVALID INPUT" , if inputs did not meet the constraints.

The input format for testing

The candidate has to write the code to accept two positive numbers separated by a new line.

- First Input line – Accept value of V.
- Second Input line- Accept value for W.

The output format for testing

- Written program code should generate two outputs, each separated by a single space character(see the example)
- Additional messages in the output will result in the failure of test case

```
v=5
w=6
if (w&1)==1 or w<2 or w<=v:
   print("INVALID INPUT")
else:
   x=((4*v) -w)//2
   print("TW={0} FW={1}".format(x,v-x))
```

40. 2022 – September Day 1 – Slot 1

Problem Statement – Given a string S(input consisting) of '*' and '#'. The length of the string is variable. The task is to find the minimum number of '*' or '#' to make it a valid string. The string is considered valid if the number of '*' and '#' are equal. The '*' and '#' can be at any position in the string.

Note : The output will be a positive or negative integer based on number of '*' and '#' in the input string.

- (*>#): positive integer
- (#>*): negative integer
- (#=*): 0

Example 1:

Input 1:

- ###*** -> Value of S

Output :

- 0 → number of * and # are equal

```
s="Hello"
a=0
b=0
for i in s:
   if i=='*':
      a+=1
   elif i=='#':
      b+=1
print(a-b)
```

41. **Day 1 Slot 2 – Question 1**

Given an integer array Arr of size N the task is to find the count of elements whose value is greater than all of its prior elements.

Note : 1st element of the array should be considered in the count of the result.

For example,

Arr[]={7,4,8,2,9}

As 7 is the first element, it will consider in the result.

8 and 9 are also the elements that are greater than all of its previous elements.

Since total of 3 elements is present in the array that meets the condition.

Hence the output = 3.

Example 1:

Input

5 -> Value of N, represents size of Arr

7-> Value of Arr[0]

4 -> Value of Arr[1]

8-> Value of Arr[2]

2-> Value of Arr[3]

9-> Value of Arr[4]

Output :

3

Example 2:

5 -> Value of N, represents size of Arr

3 -> Value of Arr[0]

4 -> Value of Arr[1]

5 -> Value of Arr[2]
8 -> Value of Arr[3]
9 -> Value of Arr[4]
Output :
5
Constraints

- 1<=N<=20
- 1<=Arr[i]<=10000

```
import sys
n=int(input())
c=0
m=-sys.maxsize-1
while n:
   n-=1
   a=int(input())
   if a>m:
      m=a
      c+=1
print(c)
```

42. Day 1 Slot 2 – Question 2

A parking lot in a mall has RxC number of parking spaces. Each parking space will either be empty(0) or full(1). The status (0/1) of a parking space is represented as the element of the matrix. The task is to find index of the prpeinzta row(R) in the parking lot that has the most of the parking spaces full(1).

Note :
RxC- Size of the matrix
Elements of the matrix M should be only 0 or 1.
Example 1:
Input :
3 -> Value of R(row)
3 -> value of C(column)
[0 1 0 1 1 0 1 1 1] -> Elements of the array M[R][C] where each element is separated by new line.
Output :
3 -> Row 3 has maximum number of 1's
Example 2:
input :
4 -> Value of R(row)
3 -> Value of C(column)
[0 1 0 1 1 0 1 0 1 1 1 1] -> Elements of the array M[R][C]
Output :
4 -> Row 4 has maximum number of 1's

```
r=int(input())
c=int(input())
sum=0
m=0
id=0
for i in range(r):
```

```
  for j in range(c):
    sum+=int(input())
  if sum>m:
    m=sum
    id=i+1
  sum=0
print(id)
```

43. Day 2 Slot 1 – Question 1

A party has been organised on cruise. The party is organised for a limited time(T). The number of guests entering (E[i]) and leaving (L[i]) the party at every hour is represented as elements of the array. The task is to find the maximum number of guests present on the cruise at any given instance within T hours.

Example 1:

Input :

- 5 -> Value of T
- [7,0,5,1,3] -> E[], Element of E[0] to E[N-1], where input each element is separated by new line
- [1,2,1,3,4] -> L[], Element of L[0] to L[N-1], while input each element is separate by new line.

Output :

8 -> Maximum number of guests on cruise at an instance.

Explanation:

1st hour:

Entry : 7 Exit: 1

No. of guests on ship : 6

2nd hour :

Entry : 0 Exit : 2

No. of guests on ship : 6-2=4

Hour 3:

Entry: 5 Exit: 1

No. of guests on ship : 4+5-1=8

Hour 4:

Entry : 1 Exit : 3

No. of guests on ship : 8+1-3=6

Hour 5:

Entry : 3 Exit: 4

No. of guests on ship: 6+3-4=5

Hence, the maximum number of guests within 5 hours is 8.

Example 2:

Input:

4 -> Value of T

[3,5,2,0] -> E[], Element of E[0] to E[N-1], where input each element is separated by new line.

[0,2,4,4] -> L[], Element of L[0] to L[N-1], while input each element in separated by new line

Output:

6

Cruise at an instance

Explanation:

Hour 1:

Entry: 3 Exit: 0
No. of guests on ship: 3
Hour 2:
Entry : 5 Exit : 2
No. of guest on ship: 3+5-2=6
Hour 3:
Entry : 2 Exit: 4
No. of guests on ship: 6+2-4= 4
Hour 4:
Entry: 0 Exit : 4
No. of guests on ship : 4+0-4=0
Hence, the maximum number of guests within 5 hours is 6.
The input format for testing
The candidate has to write the code to accept 3 input.
First input- Accept value for number of T(Positive integer number)
Second input- Accept T number of values, where each value is separated by a new line.
Third input- Accept T number of values, where each value is separated by a new line.
The output format for testing
The output should be a positive integer number or a message as given in the problem statement(Check the output in Example 1 and Example 2)

Constraints:

- 1<=T<=25
- 0<= E[i] <=500
- 0<= L[i] <=500

```
import java.util.*;
class Main
{
   public static void main (String[]args)
   {
      Scanner sc = new Scanner (System.in);
      int t = sc.nextInt ();
      int e[] = new int[t];
      int l[] = new int[t];
      for (int i = 0; i < t; i++)
         e[i] = sc.nextInt ();

         for (int i = 0; i < t; i++)
            l[i] = sc.nextInt ();

            int max = 0, sum = 0;
         for (int i = 0; i < t; i++)
         {
            sum += e[i] - l[i];
            max = Math.max (sum, max);
         }
      System.out.println (max);
}
}
```

44. Day 2 Slot 1 – Question 2

At a fun fair, a street vendor is selling different colours of balloons. He sells N number of different colours of balloons (B[]). The task is to find the colour (odd) of the balloon which is present odd number of times in the bunch of balloons.

Note: If there is more than one colour which is odd in number, then the first colour in the array which is present odd number of times is displayed. The colours of the balloons can all be either upper case or lower case in the array. If all the inputs are even in number, display the message "All are even".

Example 1:

- 7 -> Value of N
- [r,g,b,b,g,y,y] -> B[] Elements B[0] to B[N-1], where each input element is sepārated by new line.

Output :

- r -> [r,g,b,b,g,y,y] -> "r" colour balloon is present odd number of times in the bunch.

Explanation:

From the input array above:

- r: 1 balloon
- g: 2 balloons
- b: 2 balloons
- y : 2 balloons

  Hence , r is only the balloon which is odd in number.

Example 2:

Input:

- 10 -> Value of N
- [a,b,b,b,c,c,c,a,f,c] -> B[], elements B[0] to B[N-1] where input each element is separated by new line.

Output :

b-> 'b' colour balloon is present odd number of times in the bunch.

Explanation:

From the input array above:

- a: 2 balloons
- b: 3 balloons
- c: 4 balloons
- f: 1 balloons

Here, both 'b' and 'f' have odd number of balloons. But 'b' colour balloon occurs first.

Hence , b is the output.

Input Format for testing

The candidate has to write the code to accept: 2 input

- First input: Accept value for number of N(Positive integer number).
- Second Input : Accept N number of character values (B[]), where each value is separated by a new line.

Output format for testing

The output should be a single literal (Check the output in example 1 and example 2)

Constraints:

- 3<=N<=50
- B[i]={{a-z} or {A-Z}}

```
import java.util.*;
```

```
class Main
{
   public static void main (String[]args)
   {
      Scanner sc = new Scanner (System.in);
      int n = sc.nextInt ();
      char arr[] = new char[n];
      for (int i = 0; i < n; i++)
         arr[i] = sc.next ().charAt (0);
      int lower[] = new int[26];
      int upper[] = new int[26];
      for (int i = 0; i < n; i++)
      {
         if ((arr[i] >= 'A') && (arr[i] <= 'Z'))
         upper[arr[i] - 'A']++;
         else if ((arr[i] >= 'a') && (arr[i] <= 'z'))
         lower[arr[i] - 'a']++;
      }
      boolean flag = false;
      char ch = '\0';
      for (int i = 0; i < n; i++)
      {
         if ((arr[i] >= 'A') && (arr[i] <= 'Z'))
         {
            if (upper[arr[i] - 'A'] % 2 == 1)
               {
                  ch = (char) (arr[i]);
                  flag = true;
                  break;
               }
         }
         else if ((arr[i] >= 'a') && (arr[i] <= 'z'))
         {
            if (lower[arr[i] - 'a'] % 2 == 1)
            {
               ch = (char) (arr[i]);
               flag = true;
               break;
            }

         }

      }
      if (flag == true)
         System.out.println (ch);
      else
         System.out.println ("All are even");
   }
}
```

45. There is a JAR full of candies for sale at a mall counter. JAR has the capacity N, that is JAR can contain maximum N candies when JAR is full. At any point of time. JAR can have M number of Candies where M<=N. Candies are served to the customers. JAR is never remain empty as when last k candies are left.

JAR if refilled with new candies in such a way that JAR get full.
Write a code to implement above scenario. Display JAR at counter with available number of candies. Input should be the number of candies one customer can order at point of time. Update the JAR after each purchase and display JAR at Counter.
Output should give number of Candies sold and updated number of Candies in JAR.
If Input is more than candies in JAR, return: “INVALID INPUT”

Given,

N=10, where N is NUMBER OF CANDIES AVAILABLE

K =< 5, where k is number of minimum candies that must be inside JAR ever.

Example 1:(N = 10, k =< 5)

Input Value

3

Output Value

NUMBER OF CANDIES SOLD : 3

NUMBER OF CANDIES AVAILABLE : 7

Example : (N=10, k<=5)

Input Value

0

Output Value

INVALID INPUT NUMBER OF

CANDIES LEFT : 10

```
total_candies = 10
no_of_candies = int(input())
if no_of_candies in range(1, 6):
        print('No. of Candies Sold:',no_of_candies)
        print('No. of Candies Left:',total_candies-no_of_candies)
else:
        print("Invalid Input")
        print('No. of Candies Left:',total_candies)
```

46. Selection of MPCS exams include a fitness test which is conducted on ground. There will be a batch of 3 trainees, appearing for running test in track for 3 rounds. You need to record their oxygen level after every round. After trainee are finished with all rounds, calculate for each trainee his average oxygen level over the 3 rounds and select one with highest oxygen level as the most fit trainee. If more than one trainee attains the same highest average level, they all need to be selected.

Display the most fit trainee (or trainees) and the highest average oxygen level.

Note:

- **The oxygen value entered should not be accepted if it is not in the range between 1 and 100.**
- If the calculated maximum average oxygen value of trainees is below 70 then declare the trainees as unfit with meaningful message as “All trainees are unfit.
- Average Oxygen Values should be rounded.

Example 1:

INPUT VALUES

95

92

95

92

90
92
90
92
90

OUTPUT VALUES

Trainee Number : 1

Trainee Number : 3

Note:

Input should be 9 integer values representing oxygen levels entered in order as

Round 1

- Oxygen value of trainee 1
- Oxygen value of trainee 2
- Oxygen value of trainee 3

Round 2

- Oxygen value of trainee 1
- Oxygen value of trainee 2
- Oxygen value of trainee 3

Round 3

- Oxygen value of trainee 1
- Oxygen value of trainee 2
- Oxygen value of trainee 3

Output must be in given format as in above example. For any wrong input final output should display "INVALID INPUT"

```
trainee = [[],[],[],[]]
for i in range(3):
        for j in range(3):
                trainee[i].append(int(input()))
                if (trainee[i][-1]) not in range(1,101):
                        print("invalid input")
for i in range(3):
        trainee[3].append((trainee[2][i]+trainee[1][i]+trainee[0][i])//3)
maximum = max(trainee[3])
for i in range(3):
        if trainee[3][i] < 70 :
                print("Trainee {0} is unfit".format(i+1))
        elif trainee[3][i] == maximum:
                print("Trainee Number: ",i+1)
```

47. Problem Statement

A washing machine works on the principle of Fuzzy System, the weight of clothes put inside it for washing is uncertain But based on weight measured by sensors, it decides time and water level which can be changed by menus given on the machine control area.

For low level water, the time estimate is 25 minutes, where approximately weight is between 2000 grams or any nonzero positive number below that.

For medium level water, the time estimate is 35 minutes, where approximately weight is between 2001 grams and 4000 grams.

For high level water, the time estimate is 45 minutes, where approximately weight is above 4000 grams.
Assume the capacity of machine is maximum 7000 grams
Where approximately weight is zero, time estimate is 0 minutes.
Write a function which takes a numeric weight in the range [0,7000] as input and produces estimated time as output is: “OVERLOADED”, and for all other inputs, the output statement is “INVALID INPUT”.
Input should be in the form of integer value –
Output must have the following format –
Time Estimated: Minutes

Example:

Input value

2000

Output value

Time Estimated: 25 minutes

Solution

```
n = int(input())
if n==0:
   print("Time Estimated : 0 Minutes")
elif n in range(1,2001):
   print("Time Estimated : 25 Minutes")
elif n in range(2001,4001):
   print("Time Estimated : 35 Minutes")
elif n in range(4001,7001):
   print("Time Estimated : 45 Minutes")
else:
   print("INVALID INPUT")
```

48. Problem Statement

The Caesar cipher is a type of substitution cipher in which each alphabet in the plaintext or messages is shifted by a number of places down the alphabet.
For example,with a shift of 1, P would be replaced by Q, Q would become R, and so on.
To pass an encrypted message from one person to another, it is first necessary that both parties have the ‘Key’ for the cipher, so that the sender may encrypt and the receiver may decrypt it.
Key is the number of OFFSET to shift the cipher alphabet. Key can have basic shifts from 1 to 25 positions as there are 26 total alphabets.
As we are designing custom Caesar Cipher, in addition to alphabets, we are considering numeric digits from 0 to 9. Digits can also be shifted by key places.
For Example, if a given plain text contains any digit with values 5 and keyy =2, then 5 will be replaced by 7, “-”(minus sign) will remain as it is. Key value less than 0 should result into “INVALID INPUT”

Example 1:

Enter your PlainText: All the best
Enter the Key: 1
The encrypted Text is: Bmm uif Cftu
Write a function CustomCaesarCipher(int key, String message) which will accept plaintext and key as input parameters and returns its cipher text as output.

```
def ceaser(text,key):
```

```
  result = ""
 # transverse the plain text
  for i in range(len(text)):
     char = text[i]
    # Encrypt uppercase characters in plain text
     if (char.isupper()):
        result += chr((ord(char) + key-65) % 26 + 65)
    # Encrypt lowercase characters in plain text
     elif (char.islower()):
        result += chr((ord(char) + key - 97) % 26 + 97)
     elif(char.isdigit()):
        result += str(int(char) + key)
     elif(char == '-'):
        result += '-'
     elif (char.isspace()):
        result += " "
  return result
#check the above function
text = input("Enter your plain text:")
key = int(input("Enter the key:"))
print(ceaser(text,key))
```

49. Problem Statement

We want to estimate the cost of painting a property. Interior wall painting cost is Rs.18 per sq.ft. and exterior wall painting cost is Rs.12 per sq.ft.

Take input as

1. Number of Interior walls
2. Number of Exterior walls
3. Surface Area of each Interior 4. Wall in units of square feet

Surface Area of each Exterior Wall in units of square feet

If a user enters zero as the number of walls then skip Surface area values as User may don't want to paint that wall.

Calculate and display the total cost of painting the property

Example 1:

6
3
12.3
15.2
12.3
15.2
12.3
15.2
10.10
10.10
10.00
Total estimated Cost : 1847.4 INR

Note: Follow in input and output format as given in above example

```
interior_walls = int(input())
exterior_walls = int(input())
if interior_walls:
   int_walls = []
   for i in range(interior_walls):
      int_walls.append(float(input()))
if exterior_walls:
ext_walls = []
for i in range(exterior_walls):
ext_walls.append(float(input()))
if exterior_walls < 0 or interior_walls < 0:
print("Invalid Input")
exit()
if exterior_walls and interior_walls:
print("Total estimated Cost : ",(sum(int_walls)*18+sum(ext_walls)*12),"INR")
else:
if exterior_walls:
print("Total estimated Cost : ",sum(ext_walls)*12,"INR")
elif interior_walls:
print("Total estimated Cost : ",sum(int_walls)*18,"INR")
else:
print("Total estimated Cost : 0.0 INR")
```

50. Problem Statement

A City Bus is a Ring Route Bus which runs in circular fashion.That is, Bus once starts at the Source Bus Stop, halts at each Bus Stop in its Route and at the end it reaches the Source Bus Stop again.

If there are n number of Stops and if the bus starts at Bus Stop 1, then after nth Bus Stop, the next stop in the Route will be Bus Stop number 1 always.

If there are n stops, there will be n paths.One path connects two stops. Distances (in meters) for all paths in Ring Route is given in array Path[] as given below:

Path = [800, 600, 750, 900, 1400, 1200, 1100, 1500]

Fare is determined based on the distance covered from source to destination stop as Distance between Input Source and Destination Stops can be measured by looking at values in array Path[] and fare can be calculated as per following criteria:

- If d =1000 metres, then fare=5 INR
- (When calculating fare for others, the calculated fare containing any fraction value should be ceiled. For example, for distance 900n when fare initially calculated is 4.5 which must be ceiled to 5)

Path is circular in function. Value at each index indicates distance till current stop from the previous one. And each index position can be mapped with values at same index in BusStops [] array, which is a string array holding abbreviation of names for all stops as-

"THANERAILWAYSTN" = "TH", "GAONDEVI" = "GA", "ICEFACTROY" = "IC", "HARINIWASCIRCLE" = "HA", "TEENHATHNAKA" = "TE", "LUISWADI" = "LU", "NITINCOMPANYJUNCTION" = "NI", "CADBURRYJUNCTION" = "CA"

Given, n=8, where n is number of total BusStops.

BusStops = ["TH", "GA", "IC", "HA", "TE", "LU", "NI","CA"]

Write a code with function getFare(String Source, String Destination) which take Input as source and destination stops(in the format containing first two characters of the Name of the Bus Stop) and calculate and return travel fare.

Example 1:

Input Values

ca

Ca

Output Values

INVALID OUTPUT

Example 2:

Input Values

NI

HA

Output Values

23.0 INR

Note: Input and Output should be in format given in example.

Input should not be case sensitive and output should be in the format INR

```
import math
def getFare(source,destination):
   route=[ [ "TH", "GA", "IC", "HA", "TE", "LU", "NI", "CA"],
   [800,600,750,900,1400,1200,1100,1500]
      ]
   fare = 0.0
   if not (source in route[0] and destination in route[0]):
      print("Invalid Input")
      exit()
   if route[0].index(source) < route[0].index(destination):
      for i in range(route[0].index(source),route[0].index(destination)+1):
         fare+=route[1][i]
   elif route[0].index(destination) < route[0].index(source):
      for i in range(route[0].index(source)+1,len(route[0])):
         fare+=route[1][i]
      for i in range(0,route[0].index(destination)+1):
         fare+=route[1][i]
   return float(math.ceil(fare*0.005))
source = input()
destination = input()
fare = getFare(source,destination)
if fare == 0:
print("Invalid Input")
else:
print(fare)
```

51. Problem Statement

There are total n number of Monkeys sitting on the branches of a huge Tree. As travelers offer Bananas and Peanuts, the Monkeys jump down the Tree. If every Monkey can eat k Bananas and j Peanuts. If total m number of Bananas and p number of Peanuts are offered by travelers, calculate how many Monkeys remain on the Tree after some of them jumped down to eat.

At a time one Monkeys gets down and finishes eating and go to the other side of the road. The Monkey who climbed down does not climb up again after eating until the other Monkeys finish eating.

Monkey can either eat k Bananas or j Peanuts. If for last Monkey there are less than k Bananas left on the ground or less than j Peanuts left on the ground, only that Monkey can eat Bananas(<k) along with the Peanuts(<j).

Write code to take inputs as n, m, p, k, j and return the number of Monkeys left on the Tree.

Where, n= Total no of Monkeys

k= Number of eatable Bananas by Single Monkey (Monkey that jumped down last may get less than k Bananas)

j = Number of eatable Peanuts by single Monkey(Monkey that jumped down last may get less than j Peanuts)

m = Total number of Bananas

p = Total number of Peanuts

Remember that the Monkeys always eat Bananas and Peanuts, so there is no possibility of k and j having a value zero

Example 1:

Input Values

20

2

3

12

12

Output Values

Number of Monkeys left on the tree:10

Note: Kindly follow the order of inputs as n,k,j,m,p as given in the above example. And output must include the same format as in above example(Number of Monkeys left on the Tree:)

For any wrong input display INVALID INPUT

```
#include <stdio.h>
int main ()
{
   int n, k, j, m, p;
   float atebanana = 0.0, atepeanut = 0.0;
   scanf ("%d %d %d %d %d", &n, &k, &j, &m, &p);
   if (n < 0 || k < 0 || j < 0 || m < 0 || p < 0)
   {
      printf ("INVALID INPUT");
   }
   else
   {
      if (k > 0)
      {
         atebanana = (float) (m / k);
                  m = m % k;
            }
      if (j > 0)
            {
         atepeanut = (float) (p / j);
                  p = p % j;
            }
      n = n - atebanana - atepeanut;
      if ((m != 0) || (p != 0))
               n = n - 1;
      printf ("Number of Monkeys left on the Tree:%d", n);
   }
   return 0;
}
```

52. Problem Statement

Chain Marketing Organization has has a scheme for income generation, through which its members generate income for themselves. The scheme is such that suppose A joins the scheme and makes R and V to join this scheme then A is Parent Member of R and V who are child Members. When any member joins the scheme then the parent gets total commission of 10% from each of its child members.
Child members receive commission of 5% respectively. If a Parent member does not have any member joined under him, then he gets commission of 5%.
Take name of the members joining the scheme as input.
Display how many members joined the scheme including parent member.Calculate the Total commission gained by each members in the scheme. The fixed amount for joining the scheme is Rs.5000 on which commission will be generated

SchemeAmount = 5000

Example 1: When there are more than one child members

Input : (Do not give input prompts.Accept values as follows.)

Amit //Enter parent Member as this
Y //Enter Y if Parent member has child members otherwise enter N
Rajesh,Virat //Enter names of child members of Amit in comma separated

Output:(Final Output must be in format given below.)

TOTAL MEMBERS:3
COMISSION DETAILS
Amit: 1000 INR
Rajesh :250 INR
Virat: 250 INR

Example 2: When there is only one child member in the hierarchy

Input :

Amit
Y
Rajesh

Output:

Total Members: 2
Comission Details
Amit: 500 INR
Rajesh: 250 INR

```
parent = input()
Yes_No = input()
if Yes_No == "N":
   print("TOTAL MEMBERS:1\nCOMMISSION DETAILS\n{0}: 250 INR".format(parent))
elif Yes_No == "Y":
   child=list(map(str,input().split(",")))
   print("TOTAL MEMBERS:{}".format(len(child)+1))
   print("COMMISSION DETAILS \n{0}:{1} INR".format(parent,len(child)*500))
   for i in child:
      print("{0}:250 INR".format(i))
```

53. Problem Statement

FULLY AUTOMATIC VENDING MACHINE – dispenses your cuppa on just press of button. A vending machine can serve range of products as follows:

Coffee

1. Espresso Coffee
2. Cappuccino Coffee
3. Latte Coffee

Tea

1. Plain Tea
2. Assam Tea
3. Ginger Tea
4. Cardamom Tea
5. Masala Tea
6. Lemon Tea
7. Green Tea
8. Organic Darjeeling Tea

Soups

1. Hot and Sour Soup
2. Veg Corn Soup
3. Tomato Soup
4. Spicy Tomato Soup

Beverages

1. Hot Chocolate Drink
2. Badam Drink
3. Badam-Pista Drink

Write a program to take input for main menu & sub menu and display the name of sub menu selected in the following format (enter the first letter to select main menu):

Welcome to CCD

Enjoy your

Example 1:

Input:

c

1

Output

Welcome to CCD!

Enjoy your Espresso Coffee!

Example 2:

Input:

t

9

Output

INVALID OUTPUT!

```
menu = [['Espresso Coffee','Cappuucino Coffee','Latte Coffee'], ['Plain Tea',
'Assam Tea','Ginger Tea','Cardamom Tea','Masala Tea','Lemon Tea','Green Tea',

'Organic Darjeeling Tea'], ['Hot and Sour Soup','Veg Corn Soup','Tomato Soup',

'Spicy Tomato Soup'], ['Hot Chocolate Drink', 'Badam Drink',
```

```
'Badam-Pista Drink']]
m = input()
if m=='c' or m=='t' or m=='s' or m=='b':
    if m=='c':
        submenu = int(input())
        if submenu in range(3):
            print('Welcome to CCD!\nEnjoy your {}!'.format(menu[0][submenu-1]))
        else:
            print("INVALID INPUT")
    if m=='t':
        submenu = int(input())
        if submenu in range(8):
            print('Welcome to CCD!\nEnjoy your {}!'.format(menu[1][submenu-1]))
        else:
            print("INVALID INPUT")
    if m=='s':
        submenu = int(input())
        if submenu in range(4):
            print('Welcome to CCD!\nEnjoy your {}!'.format(menu[2][submenu-1]))
        else:
            print("INVALID INPUT")
    if m=='b':
        submenu = int(input())
        if submenu in range(3):
            print('Welcome to CCD!\nEnjoy your {}!'.format(menu[3][submenu-1]))
        else:
            print("INVALID INPUT")
else:
```

```
    print("INVALID INPUT!")
```

54. Problem Statement

A doctor has a clinic where he serves his patients. The doctor's consultation fees are different for different groups of patients depending on their age. If the patient's age is below 17, fees is 200 INR. If the patient's age is between 17 and 40, fees is 400 INR. If patient's age is above 40, fees is 300 INR. Write a code to calculate earnings in a day for which one array/List of values representing age of patients visited on that day is passed as input.

Note:

- Age should not be zero or less than zero or above 120
- Doctor consults a maximum of 20 patients a day
- Enter age value (press Enter without a value to stop):

Example 1:

Input

20
30
40
50
2
3
14

Output

Total Income 2000 INR

Note: Input and Output Format should be same as given in the above example.

For any wrong input display INVALID INPUT

Output Format

Total Income 2100 INR

```
age = []

for i in range(20):
    m = input()
    if m == "":
        break
    elif int(m) in range(0,120):
        age.append(int(m))
    else:
        print("INVALID INPUT")
        exit()
fees = 0
for i in age:
    if i < 17:
        fees+=200
    elif i <40:
        fees+=400
    else:
        fees+=300
print("Total Income {} INR".format(fees))
```

TEACHING BEE

55. Sum of the digits of the given positive integer number N is UNO or not.

Problem Statement

Given a positive integer number N, reduce the number of digits of N by computing the sum of all the digits to get a new number. If this new number exceeds 9, then sum the digits of this new number to get another number and continue this way until a single digit value is obtained as the 'digit sum'. The task here is to find out whether the result of the digit sum done this way is '1' or not. If the result is 1 return UNO else not.

Algorithm

Idea is to keep doing sum until sum is not less than or equal to 9

C

```
int sumOfDigits(int n) {
  int sum = 0;
  while (n > 0 || sum > 9) {
    if (n == 0) {
      n = sum;
      sum = 0;
    }
    sum += n % 10;
    n /= 10;
  }
  return sum;
}
int main() {
  int n = 235;
  if (sumOfDigits(n) == 1) {
    cout << "UNO";
  } else {
    cout << "Not UNO";
  }
  return 0;
}
```

56. Find out whether the jet plane returns to the same position from where it took off.

Problem Statement

Most modern aircrafts are equipped with an autopilot system – one of the most useful features in fighter jets. In the beta testing of autopilot mode, one of the inputs is a string of literals containing the directions, which is fed into the flight computer before the take-off. The jet plane is put on an auto-

landing mode that enables the plane to land automatically once all the directions in the string are complete. Given the directions in the string “str”, the task here is to find out whether the jet plane returns to the same position from where it took off.

- Each direction in the string changes at an interval of 1 minute(1< =i<= N), where N is the number of directions in the input string.
- The directions are North (N), South(S), West(W) and East(E).

Output:

- “Returned successfully”, if the plane returns to the starting position.
- “Not returned successfully”, if the plane does not return to the starting position.

Example 1: Input: NESW **Output**: Returned successfully

Example 2: Input: NNWESW. **Output**: Not returned successfully

Algorithm:

1. The idea is to keep track of x and y coordinate of jet plane.
2. For **'N'** 'increment y coordinate, for **'E'** increase x coordinate . Similarly for **'S' and 'W'** decrease y and x coordinate respectively.
3. Lastly check if both x and y are 0 if yes jet plane has returned successfully else no.

```
public class Main {
  static int getJetDirections(String directions) {
    int x = 0, y = 0;
    for (int i = 0; i < directions.length(); i++) {
      if (directions.charAt(i) == 'N') {
        y++;
      }
      if (directions.charAt(i) == 'E') {
        x++;
      }
      if (directions.charAt(i) == 'W') {
        x--;
      }
      if (directions.charAt(i) == 'S') {
        y--;
      }
    }
    if (x == 0 && y == 0)
```

```
        return 1;
    else
        return 0;
  }
  public static void main(String[] args) {
    String str = "NESW";
    if (getJetDirections(str) == 1)
        System.out.println("Returned successfully");
    else
        System.out.println("Not Returned successfully");
  }
}
```

57. Possible combinations of the coins that can be inserted to get rupees from the kiosk.

A 'coin vend' kiosk is installed all the major metro stations The machine allows one to obtain cash of 'R' rupees in exchange for coins. The machine operates with the following conditions:

1. Only coins of denomination 1 rupee and 2 rupee can be exchanged.
2. Coins of denomination 2 rupees should not be inserted successively twice.

The task here to find all the possible combinations of the coins that can be inserted to get rupees from the kiosk.

Input: 3 (1 + 1 + 1), (2 + 1), (1 + 2), **Output**: 6

Algorithm:

The idea is to use recursion. If 1 is inserted then we can insert 1 and 2 both. But if we have inserted 2 then again 2 cannot be inserted so only 1 can be inserted. Thus, are recursion becomes ***coinCombinations(n - 1) + coinCombinations(n - 3);*** *[As 2 comes in combination with 1 so 3 will be subtracted instead of 1*]

```
class Main {
  static int coinCombinations(int n) {
    if (n == 0) {
        return 1;
    }
    if (n == 1) {
        return 1;
    }
```

```
        if (n == 2) {
            return 2;
        }
        return coinCombinations(n - 1) + coinCombinations(n - 3);
    }
    public static void main(String[] args) {
        int n=5;
        System.out.println(coinCombinations(n));
    }
}
```

58. Find the number of students whose height is less than the height of their adjacent students.

Problem Statement

A physical education teacher asks students to assemble in a straight line for the morning assembly.Given an array of N in which each element represents the height of the student in that position. The task here is to find the number of students whose height is less than the height of their adjacent students.

Input: 35, 15, 45,25,55 **Output**: 2 (35>15<45 and 45>25<55)

Algorithm

The idea is to traverse array and compare with its left and right adjacent elements.If it is less than both the elements increment the count

```
public class Main {
    public static void main(String[] args) {
        Integer[] arr = {
            35,
            15,
            45,
            25,
            55
        };
        int count = 0;
        for (int i = 1; i < arr.length - 1; i++) {
            if (arr[i - 1] > arr[i] && arr[i + 1] > arr[i]) {
                count++;
            }
```

```
    }
    System.out.println(count);
  }
}
```

59. Find the largest number among them which does not contain 9.

Problem Statement

Rakesh once had trouble finding the numbers in a string. The numbers are distributed across a string.You need to parse only those numbers which do not contain 9. For eg, if the string contains *"hello this is alpha 5051 and 9475 TCS Coding Questions"*. You will extract *5051* and *not 9475*. Print the largest number.

Input: This is alpha 5057 and 97 **Output:** 5057

Algorithm:

1. The idea is to traverse string and check if it is a digit.
2. If digit has been encountered loop until character is found and form the number from these digits.
3. If the number doesn't contains 9 then find the maximum from current max and the number formed.

```
public class Main {
  static long ExtractNumber(String S) {
    long ans = -1;
    long result = 0;
    for (int x = 0; x < S.length(); ++x) {
      // check if it is a digit.
      if (Character.isDigit(S.charAt(x))) {
        boolean temp = true;
        result = 0;
        // Form number from digits encountered.
        while (x < S.length() && Character.isDigit(S.charAt(x))) {
          int value = (int) Integer.parseInt(S.valueOf(S.charAt(x)));
          // check if number contains 9. If yes, we don't need to consider that
number.
          if (value == 9) {
            temp = false;
          }
          result = result * 10 + value;
          ++x;
        }
```

```
            // Consider only non 9 digit numbers
            if (temp) {
                ans = Math.max(result, ans);
            }
        }
    }
    return ans;
  }
  public static void main(String[] args) {
    String s = "hello this is alpha 5051 and 9475";
    System.out.println(ExtractNumber(s));
  }
}
```

60. Order Management

Problem Statement:

A store has different categories of products in stock as shown below.
Item Number=[101, 102, 103, 108] **Price**=[42, 50, 500, 40] **Stock** =[10, 20, 15, 16]

User Inputs two values:

- Item number for item which user wish to buy
- Quantity for the item entered above

1. **If quantity is less than stock and item is available display a notification message showing** Output: Total price in float with precision and updated stock for item after after purchase
2. **If the quantity and stocks less than quantity entered by the user while placing order, then** Output: NO STOCK and quantity left
3. **If user enter character as input for item number and quantity or enter item number which is not available** Output: INVALID INPUT

Algorithm

The idea is to make hashmap with key as item number and value as price and stock. Check if item number is present or not. If present return required result or else return Invalid input.

```
import java.util.*;
class Item {
```

```
    int price;
    int stock;
    Item(int p, int s) {
        this.price = p;
        this.stock = s;
    }
}
public class Main {
    public static void main(String[] args) {
        int[] itemNumber = {
            101,
            102,
            103,
            108
        };
        int[] price = {
            42,
            50,
            500,
            40
        };
        int[] stock = {
            10,
            20,
            15,
            16
        };
        HashMap < Integer, Item > itemStore = new HashMap < > ();
        for (int i = 0; i < itemNumber.length; i++) {
            Item item = new Item(price[i], stock[i]);
            itemStore.put(itemNumber[i], item);
        }
        int user_input = 101;
        int user_quantity = 2;
        if (itemStore.containsKey(user_input)) {
            Item item = itemStore.get(user_input);
```

```
        if (item.stock < user_quantity) {
            System.out.println("No Stock");
            System.out.println(item.stock + " Quantity left");
        } else {
            System.out.println("Total Price: " + item.price * user_quantity);
            System.out.println(item.stock - user_quantity + " Quantity left");
            Item itemUpdated = new Item(item.price, item.stock - user_quantity);
            itemStore.put(user_input, itemUpdated);
        }
    } else {
        System.out.println("Invalid Input");
    }
  }
}
```

61. **Split String into three palindromic substrings**

Problem Statement:

Given an input string word, split the string into exactly 3 palindromic substrings. Working from left to right, choose the smallest split for the first substring that still allows the remaining word to be split into 2 palindromes.

Input: *str = "ababbcb"*
Output: *a bab bcb*
Explanation: *Possible splits are {"aba", "b", "bcb"} and {"a", "bab", "bcb"}. Since, {"a", "bab", "bcb"} has splits at earlier indices, it is the required answer.*

Algorithm:

The idea is to run two loops and checking palindrome property for every 3 substrings.

Implementation

```
public class Main {
  // Check If String is Pallindrome
  static boolean checkPallindrome(String str) {
    int i = 0, j = str.length() - 1;
    while (i < j) {
      if (str.charAt(i) != str.charAt(j))
        return false;
      i++;
```

```
        j--;
    }
    return true;
  }
  public static void main(String[] args) {
    String s1, s2, s3;
    String s = "ababbcb";
    int n = s.length();
    for (int i = 1; i < n - 1; i++) {
      // Substring one
      s1 = s.substring(0, i);
      // If substring is pallindrome
      if (checkPallindrome(s1)) {
        for (int j = 1; j < n - i; j++) {
          // Check if Substring two and Substring three is pallindrome
          s2 = s.substring(i, i + j);
          s3 = s.substring(i + j, n);
          if (checkPallindrome(s2) && checkPallindrome(s3)) {
            System.out.println(s1);
            System.out.println(s2);
            System.out.println(s3);
            return;
          }
        }
      }
    }
    System.out.println("Not Possible");
    return;
  }
}
```

CodeWindow

62. Bastin once had trouble finding the numbers in a string. The numbers are distributed in a string across various test cases. There are various numbers in each test case you need to find the number in each test case. Each test case has various numbers in sequence. You need to find only those numbers which do not contain 9. For eg, if the string contains “hello this is alpha 5051 and 9475”.You will extract 5051 and not

9475. You need only those numbers which are consecutive and you need to help him find the numbers. Print the largest number.

Note: Use long long for storing the numbers from the string.

Input Format :

The first line consists of T test cases and next T lines contain a string.

Output Format:

For each string output the number stored in that string if various numbers are there print the largest one. If a string has no numbers print -1.

Constraints:

1 ≤ T ≤ 100

1 ≤ |S| ≤ 10000

Sample Input:

1

This is alpha 5057 and 97

Sample Output:

5057

```
include<bits/stdc++.h>
using namespace std;
int main()
{
int t;
cin >> t;
cin.ignore();
while(t--)
{
string s;
getline(cin, s);
int n = s.length();
int n9 = 0;
string res="", num = "";
for(int i = 0; i < n; i++)
{
n9 = 0;
num = "";
while(s[i] >= '0' && s[i] <= '9')
{
if(s[i] == '9')
n9 = 1;
```

```
num = num + s[i];
i++;
}
if(!n9 && num != "")
{
long long a = stoll(num);
long long b = -1;
if(res != "")
b = stoll(res);
if(a> b)
res = num;
}
}
if(res == "")
cout << "-1";
else
cout <<res << endl;
}
return 0
//end
```

FACE PREP

63. A Sober Walk

Our hoary culture had several great persons since time immemorial and king Vikramaditya's nava ratnas (nine gems) belongs to this ilk. They are named in the following shloka:

tcs-coding-questions-images.webp

Among these, Varahamihira was an astrologer of eminence and his book Brihat Jataak is recokened as the ultimate authority in astrology. He was once talking with Amarasimha, another gem among the nava ratnas and the author of the Sanskrit thesaurus, Amarakosha. Amarasimha wanted to know the final position of a person, who starts from the origin 0 0 and travels per the following scheme.

- He first turns and travels 10 units of distance
- His second turn is upward for 20 units
- The third turn is to the left for 30 units

- The fourth turn is downward for 40 units
- The fifth turn is to the right(again) for 50 units

… And thus he travels, every time increasing the travel distance by 10 units.

Constraints:

2<=n<=1000

Input:

3
OUTPUT
-20 20

```
n = int(input())
c = 'R'
x,y=0,0
for i in range(n):
   if c=='R':
      x = abs(x) + 10;
      y = abs(y);
      c ='U';
   elif c=='U':
      y = y + 20;
      c = 'L';
   elif c=='L':
      x = -(x + 10);
      c = 'D';
   elif c=='D':
      y = -(y);
      c = 'R';
print(x,y)
```

64. **#3: Word is the key**

One programming language has the following keywords that cannot be used as identifiers:
break, case, continue, default, defer, else, for, func, goto, if, map, range, return, struct, type, var
Write a program to find if the given word is a keyword or not
Input #1:
defer
Output:
defer is a keyword
Input #2:
While
OUTPUT
while is not a keyword

MPCS

```
keyword = {"break", "case", "continue", "default", "defer", "else", "for",
```

```
"func", "goto", "if", "map", "range", "return", "struct", "type", "var"}

input_var = input()
if input_var in keyword:
   print(input_var+ " is a keyword")
else:
   print(input_var+ " is not a keyword")
```

65. **#8: Minting Mints**

Problem statement:

It was one of the places, where people need to get their provisions only through fair price ("ration") shops. As the elder had domestic and official work to attend to, their wards were asked to buy the items from these shops. Needless to say, there was a long queue of boys and girls. To minimize the tedium of standing in the serpentine queue, the kids were given mints. I went to the last boy in the queue and asked him how many mints he has. He said that the number of mints he has is one less than the sum of all the mints of kids standing before him in the queue. So I went to the penultimate kid to know how many minutes she has.

She said that if I add all the mints of kids before her and subtract one from it, the result equals the mints she has. It seemed to be a uniform response from everyone. So, I went to the boy at the head of the queue consoling myself that he would not give the same response as others. He said, "I have four mints". Given the number of the first kid's mints (n) and the length (len) of the queue as input, write a program to display the total number of mints with all the kids.

constraints:

2<n<10

1<len<20

Input#1:

4 2

Output:

7

Input#2:

14 4

OUTPUT

105

```
s,n = map(int,input().split())
sum=s
for i in range(1,n):
   prev=sum-1
   sum+=prev
print(sum)
```

66. **Consider the following series:** 1,1,2,3,4,9,8,27,16,81,32,243,64,729,128,2187....
This series is a mixture of 2 series : all the odd terms in this series form a geometric series and all the even terms form yet another geometric series.
Write a program to find the Nth term in the series.

Explantion : The value N is a positive integer that should be read from STDIN. The Nth term that is calculated by the program should be written to STDOUT. Other than the value of the nth term, no other character/string or message should be written to STDOUT.
For example: if N=16, the 16th term in the series is 2187, so only value 2187 should be printed to STDOUT.
You can assume that N will not exceed 30.

```
// Find the Nth term in the series
#include <stdio.h>
#include <math.h>

int main()
{
   int n;
   printf("Enter the Nth term");
   scanf("%d", &n); //Read from STDIN
   if(n % 2 == 1)
      {
         int a = 1;
         int r = 2;
         int term_in_series = (n+1)/2;
         int res = pow(2, term_in_series-1);
         printf("%d ",res);
      }
   else
      {
         int a = 1;
         int r = 3;
         int term_in_series = n/2;
         int res = pow(3, term_in_series-1);
         printf("%d ",res);
      }
   return 0;
}
```

Output
Enter the Nth term : 8
27

67. **Oddly Even:**
Given a maximum of 100 digit numbers as input, find the difference between the sum of odd and even

position digits.

Case 1:
Input: 4567
Expected Output: 2
Explantion : Odd positions are 4 and 6 as they are pos: 1 and pos: 3, both have sum 10. Similarly, 5 and 7 are at even positions pos: 2 and pos: 4 with sum 12. Thus, difference is 12 - 10 = 2

```
// Find the Nth term in the series
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

int main()
{
   int a = 0,b = 0,i = 0, n;
   char num[100];

   printf("Enter the number:");
   scanf("%s",num);    //get the input up to 100 digit
   n = strlen(num);
   while(n>0)
   {
      if(i==0)        //add even digits when no of digit is even and vise versa
      {
         a+=num[n-1]-48;
         n--;
         i=1;
      }
      else          //add odd digits when no of digit is even and vice versa
      {
         b+=num[n-1]-48;
         n--;
         i=0;
      }
   }
   printf("Difference between the sum of odd and even position digits is %d",abs(a-b)); //print the
difference of odd and even

   return 0;
}
```

Output
Enter the number:4567
Difference between the sum of odd and even position digits is :2

68. **Consider the following series:** 0,0,2,1,4,2,6,3,8,4,10,5,12,6,14,7,16,8
This series is a mixture of 2 series, all the odd terms in this series form even numbers in ascending order

and every even terms is derived from the previous term using the formula (x/2).Write a program to find the nth term in this series.

Explantion : The value of n is a positive integer that should be read from STDIN and the nth term that is to be calculated by the program should be written to STDOUT. Other than the value of the nth term, no other characters /strings or message should be written to STDOUT.
For example: If n=10, the 10th term in the series is to be derived from the 9th term in the series. The 9th term is 8 so the 10th term is (8/2)=4. Only the value 4 should be printed to STDOUT.
You can assume that the n will not exceed 20,000.

```
// Find the Nth term in the series
#include<stdio.h>

int main()
{
   int i, n, a=0, b=0;
   printf("enter number : ");
   scanf("%d",&n);

   for(i=1;i<=n;i++)
   {
      if(i%2!=0)
      {
         if(i>1)
            a = a + 2;
      }
      else
      {
         b = a/2;
      }
   }

   if(n%2!=0)
   {
      printf("%d",a);
   }
   else
   {
      printf("%d",b);
   }

   return 0;
}
```

Output
Enter the number:10
4

69. **SWEET SEVENTEEN**:Given a maximum of four digit to the base 17 (10 - A, 11 - B, 12 - C, 13 - D .... 16 - G} as input, output its decimal value.

Case 1:

Input - 1A

Expected Output - 27

Explantion : The value A will be multiplied by 17^0 and value 1 will be multiplied by 17^1.

=> ((1*17^1) + (A*17^0))

=> ((1*17^1) + (10*17^0)) //As A equals 10

=> 17 + 10 => 27

So the output will be 27

```
// SWEET SEVENTEEN
#include <stdio.h>
#include <math.h>
#include <string.h>

int main(){

  char hex[17];
  long long decimal, place;

  int i = 0, val, len;
  decimal = 0;
  place = 1;

  scanf("%s",hex);

  len = strlen(hex);
  len--;

  for(i = 0;hex[i]!='\0';i++)
  {
    if(hex[i]>='0'&& hex[i]<='9'){

      //48 to 57 are ascii values of 0 - 9
      //say value is 8 its ascii will be 56
      //val = hex[i] - 48 => 56 - 48 => val = 8

      val = hex[i] - 48;
    }
    else if(hex[i]>='a'&& hex[i]<='g'){

      //97 to 103 are ascii values of a - g
      //say value is g its ascii will be 103
      //val = hex[i] - 97 + 10 => 103 - 97 + 10=> val = 16
      //10 is added as g value is 16 not 6 or a value is 10 not 0
```

```
    val = hex[i] - 97 + 10;
   }
   else if(hex[i]>='A'&& hex[i]<='G'){

    //similarly, 65 to 71 are values of A - G
    val = hex[i] - 65 + 10;
   }

   decimal = decimal + val * pow(17,len);
   len--;
  }

  printf("%lld",decimal);

  return 0;
}
```

Output

1A

27

70. **Q1) Given an array of integers where every element appears even number of times except one element which appears odd number of times, write a program to find that odd occurring element in O(log n) time. The equal elements must appear in pairs in the array but there cannot be more than two consecutive occurrences of an element.**

For example :

3

2 3 2

It doesn't have equal elements appear in pairs

7

1 1 2 2 2 3 3

It contains three consecutive instances of an element.

5

2 2 3 1 1

It is valid and the odd occurring element present in it is 3.

Enter only valid inputs.

Sample Input :

5

2 2 3 1 1

Sample Output :

3.

PYTHON

```
n = int(input())
a = list(map(int,input().split()))
left=0
right=n-1
if a[0] != a[1]:
   print(a[0])
elif a[n-1] != a[n-2]:
   print(a[n-1])
else:
   while left <= right:
      mid = ((right-left)//2)+left
      pre = mid-1
      nxt = mid+1
      if (a[pre] != a[mid])  and (a[nxt] != a[mid]):
         print(a[mid])
         break
      elif mid%2==0 :
         if a[pre] == a[mid] :
            right = mid - 1
         else :
            left = mid + 1
      else :
         if a[pre] == a[mid] :
            left = mid + 1
         else :
            right = mid - 1;
```

71. **Q2) Given an array of integers and a sum, the task is to count all subsets of given array with sum equal to given sum.**

Input :

The first line of input contains an integer T denoting the number of test cases. Then T test cases follow. Each test case contains an integer n denoting the size of the array. The next line contains n space separated integers forming the array. The last line contains the sum.

Output :

Count all the subsets of given array with sum equal to given sum.

NOTE: Since result can be very large, print the value modulo 109+7.

Constraints :

1<=T<=100

1<=n<=103

1<=a[i]<=103

1<=sum<=103

Example :

Input :

2
6
2 3 5 6 8 10
10
5
1 2 3 4 5
10

Output :

3
3

Explanation :

Testcase 1: possible subsets : (2,3,5) , (2,8) and (10)
Testcase 2: possible subsets : (1,2,3,4) , (2,3,5) and (1,4,5)

PYTHON

```
def printBool(n, len):
        while n:
                if n & 1:
                        print("1 ")
                else:
                        print("0 ")
                n = n >> 1
                len -= 1

        while len:
                print("0 ")
                len -= 1
        print()

def printSubsetsCount(set, n, val):
        sum = 0
        count = 0
        for i in range(0, 1 << n):
                sum = 0

                for j in range(0, n):

                        if (i & 1 << j) > 0:
                                sum += set[j]

                if (sum == val):

                        count += 1

        if (count == 0):

                print("No subset is found")
```

```
        else:
                print(count)

t=int(input())
set = []
while t>0:
   n = int(input())
   set = list((map(int,input().strip().split())))[:n]
   Sum = int(input())
   printSubsetsCount(set, n, Sum)
   t = t-1
```

72. **Q3) Before the outbreak of corona virus to the world, a meeting happened in a room in Wuhan. A person who attended that meeting had COVID-19 and no one in the room knew about it! So everyone started shaking hands with everyone else in the room as a gesture of respect and after meeting unfortunately every one got infected! Given the fact that any two persons shake hand exactly once, Can you tell the total count of handshakes happened in that meeting?**

Input Format :

The first line contains the number of test cases T, T lines follow.

Each line then contains an integer N, the total number of people attended that meeting.

Output Format :

Print the number of handshakes for each test-case in a new line.

Constraints :

1 <= T <= 1000

0 < N < 106

Sample Input :

2

1

2

Output :

0

1

Explanation :

Case 1 : The lonely board member shakes no hands, hence 0.

Case 2 : There are 2 board members, 1 handshake takes place.

PYTHON

```
t = int(input())
while t > 0 :
   n = int(input())
   n = n-1;
   print((n*(n+1))//2)
   t = t-1
```

73. **Q4) For enhancing the book reading, school distributed story books to students as part of the Children's day celebrations. To increase the reading habit, the class teacher decided to exchange the books every weeks so that everyone will have a different book to read. She wants to know how many possible exchanges are possible.**

If they have 4 books and students, the possible exchanges are 9. Bi is the book of i-th student and after the exchange, he should get a different book, other than Bi.

B1 B2 B3 B4 – first state, before exchange of the books
B2 B1 B4 B3
B2 B3 B4 B1
B2 B4 B1 B3
B3 B1 B4 B2
B3 B4 B1 B2
B3 B4 B2 B1
B4 B1 B2 B3
B4 B3 B1 B2
B4 B3 B2 B1

Find the number of possible exchanges, if the books are exchanged so that every student will receive a different book.

Constraints

1<= N <= 1000000

Input Format

Input contains one line with N, indicates the number of books and number of students.

Output Format

Output the answer modulo 100000007.

Refer the sample output for formatting

Sample Input :

4

Sample Output :

9

PYTHON

```
mod = 100000007;
a=0
b=1
c=2
d=0
n = int(input())
if n==1 or n==2:
   print(n-1)
else :
   for i in range(1,n) :
      d = (c * (a + b)) % mod
      a = b
      b = d
      c = c + 1
   print(d)
```

74. **Q5) You are given a string A and you have to find the number of different sub-strings of the string A which are fake palindromes.**

Note:
1. Palindrome: A string is called a palindrome if you reverse the string yet the order of letters remains the same. For example, MADAM.
2. Fake Palindrome: A string is called as a fake palindrome if any of its permutations is a palindrome. For example, AAC is fake palindrome, but ACD is not.
3. Sub-string: A sub-string is a contiguous sequence (non-empty) of characters within a string.
4. Two sub-strings are considered same if their starting indices and ending indices are equal.

Input Format:
First line contains a string S

Output Format:
Print a single integer (number of fake palindrome sub-strings).

Constraints:
1 <= |S| <= 2 * 105

The string will contain only Upper case 'A' to 'Z'

Sample Input 1:
ABAB

Sample Output 1:
7

Explanation:
The fake palindrome for the string ABAB are A, B, A, B, ABA, BAB, ABAB.

Sample Input 2:
AAA

Sample output 2:
6

Explanation:
The fake palindrome for the string AAA are A, A, A, AA, AA, AAA

PYTHON
```
def countSubString(s):
        res = 0;
        for i in range(len(s)):
                x = 0;
                for j in range(i, len(s)):
                        temp = 1 << ord(s[j]) - ord('a');
```

```
                    x ^= temp;
                    if ((x & (x - 1)) == 0):
                        res += 1;
        print(res);

if __name__ == '__main__':
        str = input();
        countSubString(str);
```

TELEGRAM

JAN 31 2023

75. Given two Strings of lowercase English letters P and Q. the following operations: lengths rue if the of two strings are equal else Print the ASCII value of last letter of second string.
Count the vowels from each string and print the sum of all vowels (from both the string).
Example 1:
Input:
hello 1st string with lowercase English letters world 2nd string with lowercase English letters 5311
Output:
true Lengths of both the strings are equal
100 ASCII value of lowercase letter "d"
30 Total vowel count from both the strings
Explanation:
"hello" and "world" are two strings which has lowercase letters.
Case 1.
Length of two strings is equal (ie. 5) hence first
output will be "true"
Case 2.
Last letter of second string is "d" ASC value for
lowercase dis 100
Hence output is
100
Case 3
There are 2 vowels in hello word

```
#include <iostream>
using namespace std;
int main()
string a,b;
cin>>a>>b;
if(a.size()==b.size())
cout<<"true";
else
cout<<"false";
```

```
cout<<int(b[b.size()-1]);
int ans=0;
for(int i=0;i<a.size();i++)
{
ans++;
}
if(a[i]=='a' || a[i]=='e' || a[i]=='i' || a[i] ==^ prime o^ prime || a[i] ==^ prime u^ prime )
for (int i=0;i<b.size();i++)
if( b[i] ==^ prime a^ prime ||b[i]==^ prime e^ prime || b[i] ==^ prime i^ prime || b[i] ==^ prime 0^ prime
|| b[i] ==^ * u^ * )
ans++;
}
cout<<ans;
return 0;
```

JAN 31 2023

76. Given a binary string S of length N consisting of '0' and '1 A binary string is good if the total number of consecutive ones in a string (wherever present) are equal

For eg. "011101110" There are 3 consecutive ones present at two positions.

"011101101". Here, consecutive ones are present at three positions having length 3, 2, and 1 respectively.

You are allowed to perform two types of operations

>You can choose any index i and insert '1' at that position. After this, the length of the string will increase by 1.

> Choose any index i such that S[i] = '1' and remove S[I]. After this, the length of the string will decrease by 1

Your task is to make a given string good using the minimum number of operations

```
#include <iostream>
 using namespace std;
int main()
string a,b;
cin>>a>>b;
if(a.size()==b.size())
cout<<"true";
else
cout<<"false";
cout<<int(b[b.size()-1]);
int ans=0;
for (int i=0;i<a.size();i++)
{
ans++;
}
if(a[i]=='a' || a[i]=='e' || a[i]=='i' || a[i]=='o' || a[i]=='u')
for (int i=0;i<b.size();i++)
if(b[i]=='a'
ans++;
```

```
|| b[i]=='e' || b[i]=='i' || b[i]=='o' || b[i]=='u')
}
cout<<ans;
return 0;
```

JAN 31 2023

77. Alice had to go to play with his friends. But his brother is
not leaving. So, he thought to ask a question so that in the meantime he can go. So, The problem is as follows: He will be given Numbers n.m and k. Calculate T.(T=(nm)/k).His brother has to find the three coordinates of the XX plane (2D points) such that the area of the triangle formed by those points should be equal to T. If there is any solution print YES. else NO.
NOTE: 1 ≤x1,y1, x2,y2, x3, y3 ≤ 109
Example 1:
Input:
4 3 3
Output:
YES
Explanation:
One possible way is (1,0), (2.3) and (4.1) are the points where there area is equal to T
Example - 2
Input:
447

```
def find_triangle(n, m, k):
        T = (n* m) // k
        if Tk == n * m:
                return "YES"
        else:
                return "NO"
n, m, k = map(int, input().split())
print(find_triangle(n, m, k)
```

JAN 31 2023

78. Ninja warrior is a sports entertainment competition in which h competitors attempt to finish a series of obstacles of increasing difficulty. One of the the obstacles is Rope Climb.
Where the participant has to cross ‘N’ ropes with varying Nengths. The ropes are made up of synthetic material which is slimy in nature. The participant starts with rope 1, at each jump he pulls up the rope by 'H' feet, he pulls down by 'S' feet due to the slimy rope. Given the lengths of the ropes as array elements, the task here is to find the number of jumps he will need to make to climb all the ropes and reach the next obstacle. The is to reach the top of rope, slide down and start with the next rope till N ropes.
Example 1:
Input:
4-> Value of N
20-> Value of H
2-> Value of S (20,20,16,8}-> a[], Elements a[0]to a[N-1], where input each element is separated by new line

Output
4
Explanation:
From the input given above:
Number of rones:
4

```
def rope_climb (n, h, s, ropes):
jumps = 0
for rope in ropes:
jumps = max(jumps, (rope + s - 1) // s)
 return jumps if jumps <= h else -1
 n = int(input())
 h = int(input())
 s = int(input())
ropes = [int(input()) for i in range(n)]
print(rope_climb (n, h, s, ropes))
```

JAN 31 2023

79. Mr. Rao is relocating from place A to B. The moving truck has a maximum capacity C. There are 'N' items in the house where each item has a corresponding value (V)and weight(W). Mr.Rao has to carry only the most Valuable items whose total weight does not exceed capacity of the truck. The task here is to find those items (single or combination of items) whose total value(v) will be the maximum and their corresponding weight(w) will not exceed truck capacity (C). Here
N=No. of items
C-Maximum capacity of the truck, an integer value.
W[O to N-1]- An array consisting weight of each item
V[0 to N-1] - An array consisting value of each item

Example 1:
Input:
4 à Value of N
80 à Value of C
[10, 45, 60, 90]à Elements of array VD,where each element is separated by new line
[15, 20, 30, 40]à Elements of array W[],where each element is separated by new line

```
import java.util.*;
class Solution
{
public static int solve(int c,int w[],int val[],int n)
{
if(n==0 || c==0)
return 0;
if(c<w[n-1])
return solve(c,w,val,n-1);
else
```

```
return Math.max(solve(c,w,val,n-1),val[n-1]+solve(c-w[n-1],w,val,n-1));
}
public static void main(String[] args)
{
Scanner sc=new Scanner(System.in);
int n=sc.nextInt();
int c=sc.nextInt();
int v[]=new int[n];
int w[]=new int[n];

for(int i=0;i<n;i++)
v[i]=sc.nextInt();
for(int i=0;i<n;i++)
w[i]=sc.nextInt();
System.out.println(solve(c,w,v,n));
}
}
```

JAN 30 2023

80. A single currency note(N) and a coin(R) are given. The task is to find in how many ways can u take the value that is equal to the currency note using the coin. The coin can be used in the following ways: The value of the coin represents the maximum range, values (1, R) that can be used to make the sum equal to N. Each integer value up to the range C can be used multiple times to make the sum.

Constraints:

1 <= N <= 100

1 <= R <= 100

Example 1:

Input:

8 a Value of N

2 a Value of R

Output:

5

Maximum combinations using values up to R to get the sum equal to N

Explanation:

The various combinations the currency note N=8 can be

tarined using the values upto range 2 are

R=(1,2)

1 + 1 + 1 + 1 + 1 + 1 + 1 = 8

2 + 1 + 1 + 1 + 1 + 1 + 1 = 8

2 + 2 * 2 + 1 + 1 = 8

2 + 2 + 1 + 1 + 1 + 1 = 8

2 + 2 + 2 + 2 = 8

Hence, there are 5 ways the coin with range up to 2 can be used to get the value of the currency note i.e.8

```
N=int(input())
```

```
def Numberofways (N,K):
K=int(input())
dp=[0]*(N+1)
dp[0]=1
for row in range(1,K+1): 7 for col in range(1,N+1):
if (col>=row):
dp[col]=dp[col]+dp[col-row]
return(dp[N])
print(Numberofways (N,K))
```

JAN 30 2023

81. One of the most common and essential applications in all the mobiles is the CLOCK. The app is designed to capture time automatically according to their time zones. The display format can be 12hr or 24hr based on convenience. The task here is to convert the time given as a string in 24 hour(00:00 to 23:59) format to a 12 hour format(00:00 to12:00). The format for both input and output are H:MM:SS where HH,MM,SS denotes hours, minutes and seconds respectively The output in 12 hour format shpuld denote if the time is AM or PM.

Example 1:

Input:

15:15:22 -> Value of str

Output:

3:15:22 PM->Time in 12 hour format

Explanation:

From the input given above

Str = 15:15:22

The given time is in 24-hour format

After 12PM,

13 hours is 1:00 PM

```
def convert_time_format(time):
hours, minutes, seconds = map(int, time.split(":"))
if hours == 12:
hours = 12
am pm = "PM"
elif hour's == 0:
hours = 12
am_pm =” AM”
elif hours > 12:
hours -=12
am_pm =”PM”
else:
am_pm= "AM"
return f" (hours: 02d): (minutes:02d): (seconds: 02d} {am_pm}"
time = input()
print(convert_time_format(time))
```

JAN 30 2023

82. Given N Rupees. A liter plastic bottle of milk costs R1 Rupees and a liter of the glass bottle of milk costs R2 Rupees. But the empty glass bottle after buying can be exchanged for R3 Rupees. Find the maximum liters of milk which can be bought with N Rupees

Example-1:

Input:

10

a Value of N

11 à Value of R1 i.e. price of plastic bottle

9 à Value of R2 i.e. price of glass bottle

8 a Value of R3 ie, price of empty glass bottle

Output:

2

Explanation:

One glass bottle can be bought

in hand-1 Rs

in hand-1+8=9

total milk 1 licens

Return one glass bottle

```
def max1(b,p,g,rf);
if g-rf<p;
ans=max((b-rf)//(g-rf),9)
b-=ans* (g-rf)
ans+=b//p
print(ans)
else:
print(b//p) 9 b=int(input())
p=int(input())
g=int(input()) 12 rf-int(input())
max1(b.p.g.rf)
```

JAN 30 2023

83. User has decided to odd out the array This means, if the array contains odd number, then let it be like that. But if it contains even number of elements, then add the two iddle elements, and then make it odd. So there will always be a unique middle element

So, new middle is the sum of the two elements whose index numbers when counting from the beginning and from the end of the array differ by one. The user should finally display the array

Example 1:

Input:

5> Value of N

(10,9,5,2,5)-> a[], Elements a[0]to a[N-1],where input each element is separated by new line

Output:

(10,9,5,2,5)->a[], the middle is replaced, if N is even,

otherwise same output

Explanation:

Consider the above array, it contains 5 number of elements, which is odd, which means the middle element is always unique.

Hence the output is same as the original one.

```
def make_odd_length(arr):
n = len(arr)
if n % 2 != 0:
return arr
else:
arr[n//2-1] += arr[n//2]
return arr[:n//2] + arr[n//2+1:1
```

JAN 30 2023

84. A primary school teacher is trying creative methods to teach her students to recognise numbers. One of the methods involves providing 'N' numbers to the students. She gives another special digit(S) whose occurrences should be found in all the numbers up to N. The task here is to find the count of values containing the special digit and also display all the numbers from 0 to N comprising the special digit.

Example1:

Input:

20 >> Value of N

3 -> Value of S

Output:

3 1 3 2 >Numbers upto 20 that contains

the digit 3 and the count of numbers containing the digir 3

```
def count_special_numbers(n, s):
numbers = []
count =0
for i in range(n + 1):
if str(s) in str(i):
numbers.append(i)
count += 1
print(count)
print(*numbers)
n = int(input("Enter value of N: "))
s = int(input("Enter value of S: "))
count_special_numbers(n, s)
```

JAN 30 2023

85. A box consists of N pieces of stones with each stone having certain weights assigned to it. The task is to manage all stones into that both parts should have equal weight of stones. two-part such

Print "TRUE" if the partition is possible otherwise print "FALSE".

Example 1:

Input

4 à Value of N, represents size of Arr
2 à Value of Arr[0], represents
Weight of 1st stone
9 à Value of Arr[1], represents
Weight of 2nd stone
3 à Value of Arr[2], represents
Weight of 3rd stone
4 à Value of Arr[3], represents

```
def partition_possible (n, arr):
if sum(arr) % 2 == 1:
return "FALSE"
target = sum(arr) // 2
dp = [0] (target + 1)
dp[0] = 1
for weight in arr:
for i in range(target, weight-1, -1): dp[i] = dp[i] or dp[i - weight]
return "TRUE" if dp[target] else "FALSE"
n = 4
arr [2, 9, 11, 14]
print partition_possible(n, arr))
```

JAN 30 2023

86. Few keys the keyboards are not working properly. While typing, there are few alphabets that gets induced with multiple occurences. This is has caoused huge problem to the user.
Now they have to come up with up with a feature that remove all these duplicates alphabets within a word. Only those alphabets have to be removed which have occurred more than once exactly adjacent to each other.
Example
Input:
"abciijklma" >input string containing multiple occurences of adjacent duplciate .
output
"abcjklma" >output string without any adjacent duplicate alphabets.
Explanation:
Consider the above string, it contains 2 I's next to each other. These are the ones that need to be removed. There are also 2 a's, but they are not adjacent to each other. Hence these should not be removed from the original string.

```
#include <bits/stdc++.h>
using namespace std;
int main() {
string str;
cout &#8203; oaicite: {"index":0,"invalid_reason": "Malformed citation <
int n = str.length());
string res =””;
```

```
for (int i = 0; i < n; i++) {
if (res.find(str[i]) string::npos) {
res += str[i];
}}
cout << "String after removing repeated letters: “<< res << endl;
return 0;
```

JAN 30 2023

87. Maria is teaching her 2 years old Tina-English alphabets. Tina likes donuts a lot, and whenever she sees anything which is even a little bit circular, she says donuts.maria didn't understand in the beginning, but later she got it, that any alphabets which is enclosed shape, is Wed by Tina
Let say A, this has an enclosure of triangle. Then B, it has 2 enclosures. C has no enclosures. D has 1...and so on
So, now Maria a taught Tina a new way, that if she finds any alphabets with enclosures sum up all the enclosures For eg. the word TINA has only enclosure. The word MARIA has 3 enclosures
Given a word in all upper case, find the total number of enclosures in the entire word.

Example 1:
Input:
775031160 273
60 273
HOLLYWOOD -> Input string, S
Output:
4 Output

```
d={}
V="A"
for i in range (26):
d[chr(ord (v)+1)]=0
d["A"]=1
d["B"]=2
d["D"]=1
d["0"]=1
d["p"]=1
d["Q"]=1
d["R"]=1
m=input()
C=0
for i in m:
c+=d[i]
print(c)
```

JAN 23 2023

88. Sara has a box which has N chocolates in it. Sara decides to gift the box to Alice and Bob. Bob can have x chocolates from the box and Alice gets the remaining chocolates, y. SincE chocolates, Sara tries to calm them down by playing a game.
If Bob gets x chocolates and Alice gets y chocolates, and if x>y, Bob should put y chocolates back into the box. The same rule applies to Alice as well. This continues until one of t equal number of chocolates.

Write a program to find out the number of chocolates in the box after the game is over.
Illustrate the scenario by reading the input from STDIN and writing the output to STDOUT.
You should not write arbitrary strings while reading the input and while printing as the
Constraints:
1)N 2
11) 0 < x <= N
Input Format:
A single line of input contains two integers, N and x, separated by a single whitespace.
Output Format:
Output contains the number of chocolates left in the box at the end of game.
Sample Input 1:
104
Sample Output 1:
6
Explanation 1:
Bob gets 4 chocolates and Alice gets 6 chocolates. So Alice has to put back 4 chocolates into the box. So Bob will have 4 and Alice will have 2 chocolates. Now Bob has to put chocolates each. So, there will be 10-4-6 chocolates in the box. Therefore output is 6
Sample Input2:
174

```
a=list(map(int,input().split()))
bob=a[1]
alice=a[0]-a[1] 4- while (bob!=alice):
if (bob alice):
alice--bob
elif (bob>alice):
bob-=alice
if (alice==0 or bob==0): I
break
print(a[0]-bob-alice)
```

JAN 23 2023

89. The Principal of a school is conducting interviews for admissions and students have to take a test before the interview. Roll num examination. A naughty student removed a few roll numbers and duplicated some of the roll numbers. After an inquiry, the Princ as the Kth non-repetitive roll number from the list A of roll numbers. Can you help the principal with your program to find the Kth nonrepetitive roll number by accepting a list A of roll numbers chang
Note: The solution does exist for all the test cases.

Read the input from STDIN and print the output to STDOUT. Do not write arbitrary strings while reading the input or while printing

Constraints:

i) 1<= N <=5000

ii) 0< A <100000

Input Format:

The first line of the input contains N.

Next N lines of the input contain the roll numbers of the students. The last line of the input contains k.

Output Format:

The output contains a single integer which represents the kth non-repetitive roll number of a naughty student.

Sample Input1:

8

45623

1321

55555

23123

1321

56344

9999 11111

5

```
n=int(input())
O=[]
for i in range(n):
arr=int(input())
o.append(arr) 6 k=int(input())
p=[]
for i in o:
if o.count(i)==1
p.append(i)
print(p[k-1])
```

JAN 23 2023

90. Seeing Bob enjoying his vacation by just watching TV, his father decides to teach him the arrangement of numbers by sing tim some number written on it to Bob and asks him to arrange it from small to big. As he is a small boy, he is able to hold only fest cards among N cards.

Can you help Bob with a program to complete his task.

Read the input from STDIN and print the output to STDOUT. Do not write arbitrary strings while reading the input or while pr

Constraints:

1) N >= 3 II) N > k >= 2

Input Format:

1) The first line of input contains N, an integer denoting the number of cards. II) The second line of input contains N cards separated by a single white space. III) The third line of input contains an integer, k, up to

which the sorting needs to be done.

Output Format:
N integers separated by single white space.
Sample Input1:
10
10987654321
5
Sample Output1:
67891054321
Explanation1:
As k value is 5 so bob can hold only first 5 cards whose values are 10 9 8 7 6 so after arranging from sm So the output is 67891054321.

```
s = int(input())
arr = list(map(int,input().split()))
k = int(input())
arr = sorted(arr[:k])+arr[k:]
print(*arr)
```

JAN 22 2023

91. You are given anaray Arr of size N.
A special subarray of Arr is chosen keeping following conditions in mind:
-Let the length of the first subarray chosen be "len", the length of the second subarray chosen must be len-1,..., the length of the last subarray chosen must be 1.
- Let sum be the sum of the ith subarray, then sum, must be less than sum
- The chosen subarrays must not overlap
Find the maximum number of special subarrays that could be chosen from Arr.
Notes
It is given you are only allowed to choose subarrays in order of their Occurrence in Arr. This means that the index of all elements in the first chosen subarray will always be less than the index of all elemenrs in the second subarray chosen and so on

```
#include <iostream> #include <vector>
using namespace std;
int main() {
int n;
cin >> n;
vector<int> arr(n);
for (int i = 0; i < n; i++) {
cin>> arr[i];
}
int max_special_subarray =0;
for (int i = 0; i < n; i++) {
int sum 0;
for (int j = i; j < n; j++)
```

```
sum += arr[j];
if (sum < arr[i]) {
max_special_subarray++;
}
{
}
}
return 0;
cout << max_special_subarray << endl;
}
```

JAN 22 2023

92. Bob felt bored, so he decided to start inventing following functions! He invented the functions:
-A (S): is a function that takes a string S as a parameter and returns it reversed.
-B (S): is a function that changes the order of letters of string S randomly. .
-C (S, T): is a function that merges strings S and T in a way that keeps each string in its original order. There might many ways to merge two strings, the function chooses one randomly.

You will be given a string M. which is represented as: C (A(N), B(N)) for some string N. Find the lexicographically smallest possible string N.
Notes:
It is guaranteed that the input is valid and a string N exists.
Input Format
The first line contains a string, M. denoting the given string

```
def A(s):
return s[::-1]
def B(s):
return ''.join(random.sample(s,len(s)))
def C(s, t):
l1 = list(s)
l2 = list(t)
l3 []
while l1 and l2: if random.random()<0.5:
l3.append(l1.pop(0))
else:
l3.append(l2.pop(0))
l3.extend(l1)
l3.extend(l2)
return ''.join(l3)
def solve(m):
m = m[2:-2]
s,t = m.split(',')
if s==t:
return S
S = A(s)
```

```
t = B(t)
n =
while C(s, t)!=m:
n += random.choice('abcdefghijklmnopqrstuvwxyz'
```

JAN 22 2023

93. You are given two arrays A and B consisting of N integers numbered from 0 to (N-1) and an integer X.
It is given that initially the value of X is zero
For each j such that (0 <= j <= N) you can do the following:

Choose an index i such that (0 <= i <= (N - 1))
Calculate a value Y=(A i ^ * j+ B i ^ * (N - j)).
Add Y to X (in other words X =X+Y.)

Find the maximum value of X you can get. Since the answer may be large return it modulo 10^9+7.

Input Format

The first line contains an integer, N, denoting the number of elements in A and B. Each line i of the N subsequent lines (where 0 <= i < N) contains an integer describing
A[i]
Each line i of the N subsequent lines (where 0 <= i < N) contains an integer describing

```
# Calculate the factorial of K
factorial = 1
for i in range(1, K+1): factorial = (factorial i) % MOD
# Initialize the count of distinct strings
count = 1
# Multiply the count by the frequency of each character raised to the power
for char in char_count:
count = (count* pow(char_count[char], K, MOD)) % MOD
# Divide the count by the factorial of K
count = (count pow(factorial, MOD-2, MOD)) % MOD
return count
# Test the function
S = "aab"
K=3
print(count_distinct_strings (s, K))
```

JAN 22 2023

94. Given the dimension (MN) and elements of a square matrix, find the sum S of diagonal elements, which are repeated not more than X times ____ in the m_____ dimension is not that of a square matrix, convert it into a square matrix by inserting '1' for the missing values

Note: The diagonal starting from index (0,0) is considered

Read the input from STDIN and print the to STDOUT. Do not write arbitrary strings while reading the input or while printing as these contribute to the s

Constraints:

I) 1<=M<=10

II) 1<=N<= 10

III) X>=1

Input Format:
The first line of input contains M and N separated by a single white space. The second line of input contains M'N elements separated by a single white space. The last line of input contains X.
Output Format:
The output contains S
Sample Input1:
23
123
456
1
Sample Output1:
5

```
import numpy as np
n,m=map(int, input().split())
m=[]
for i in range(n):
m.append(list(map(int,input().split())))
x=int(input())
rows = len (m)
columns= len(m[0])
if rows > columns:
for row in m:
for i in range(rows -
row.append(1)
else:
columns):
for i in range(columns - rows): m.append([1]* columns)
s_m = np.array(m)
di=[]
nondi=[]
for i in range(len(s_m)): di.append(s_m[i][i])
SS=0
for i in range(len(s_m)):
for j in range(len(s_m)): if il-j: nondi.append(s_m[i][j])
for j in di:
if nondi.count(j)<=x:
ss+=j
print(ss)
```

JAN 21 2023

95. Arya has a crush on Sam, she wants to write him a secret letter to express her feelings, but now she is worned it will be traced back to her thought the___ Ransom Note (formed from words or letters cut

randomly from a magazine or newspaper). She found a magazine and wants to know she __ to create an untraceable replica of her ransom note.
The words in his note are case-sensitive and he must use only whole words available in the magazine. He cannot use substrings concatenation to create the Given the words in the magazine and the words in the ransom note, print Yes if he can replicate his ransom note exactly wing whole from the magazines
Note: All the special characters (ie full stop or comma) are space separated.
Read the input from STDIN and write the output to STDOUT You should not write arbitrary strings while reading the input

Constraints:

1 >= n >= 100
10>= (str) >= 1000

Input Format:

First line of input consists of an integer n representing the number of messages that Arya decided to write Second line of input consists of a random string from the magazine (Consist spaces). From third line of input consists of an n-number of strings in n-number of lines that stores the message that arya wants to type

Output Format:

N lines of output contain a string yes or no, depending on the possibility of writing a message.

Sample Input 1:

1 I can't say how much I like you and how special you are to me, but I can say my world is full of smiles when I am with you. I like you and you are special to me.

Sample Output 1:

Yes

```
def can_replicate(n, magazine, ransom_note):
 word_count = {} for word in magazine.split():
 if word in word_count: word_count [word] += 1
else:
word_count[word] = 1
if word in word_count:
for word in ransom_note.split(): return "No"
word_count[word] -= 1 if word_count[word] < 0:
else:
return "No"
return "Yes"
num_messages int (input())
magazine = input()
for in range(num_messages):
ransom note = input() print(can_replicate (num_messages, magazine, ransom note)
```

JAN 20 2023

96. A section students of a school has N students in it. The mathematics teacher has distributed the marks of an MCQ test to students the performance of the students. To improve the performance, she wanted to group the students. Each group has 2 students, intelligent st performance. She wants to try this experiment with the second-highest-performing student and second lowest performing student

Now she needs a program that accepts the marks of N students and prints the second top-performing student mark and second least program
Read the input from STDIN and print the output to STDOUT. Do not write arbitrary strings while reading the input or while printing, a

Constraints:

1) 2 <= N <= 105

II) Array may contain duplicate elements III) Marks can be negative as it is an MCQ exam with negative marking

Input Format:

The first line of input contains an integer, N. Second line of input contains marks of N Students, separated by a single white space.

Output Format:

First line of output contains the second-highest mark and the second line contains the second-lowest mark.

Sample Input 1:

5
12 17 23 59

Sample Output 1:

17
9

```
n=int(input())
1-list (map(int,input().split()))
1.sort() I
print(1[-2])
print(1[1])
```

Delloitte

97. Given a string Str, write a program to remove duplicate characters from Str by following the below points.
While eliminating the duplicate characters retain the first character of the duplicate characters
Space must be excluded while eliminating the duplicate characters

Constraints:

1) Length of Str< 100
2) Any whitespace present in Str should not be removed

Input Format:

The input contains Str.

Output Format:

The output contains the modified string.
The first occurence of the duplicate character remains as is, rest should be removed from Str

Sample Input1: Hello There
Sample Output1: Helo Thr

Explanation: In Str, 'e' and 'l' have duplicate characters. Thus, these characters are removed and the output is "Helo Thr".

Sample Input2: call taxi
Sample Output2: cal txi

JAN 20 2023

98. Given a list A of N integers, write a program to square all the numbers in the list and print their sum Read the input from STDIN and print the output to STDOUT. Do not write arbitrary strings while reading the input or while printing, as these contribute to the standard output

Constraints:
1 <= N <= 10000
0 <= Ai <= 10, where i=0,1… N-1

Input Format:
The first line of input contains an integer, N, where N is the size of the list. The second line of input contains N integers, each separated by a single white space.

Output Format:
The output contains the sum

Sample Input 1:
3
123

Sample output 1:
14

Explanation1:
12+22+32-1+4+9=14

Sample Input 2:
4
3517

Sample Output 2:
84

Explanation2:
32+52 +12+72-9+25+1+49=84

```
n=int(input())
arr=list(map(int, input().strip().split()))
sum1=0
for i in arr:
sum1+=i*i
print(sum1)
```

JAN 20 2023

99. Devin has land which is a perfect square shape. He wants to try a different pattern of cultivation. He divided his complete land Into NxN eq cells where N is the number of partitions he planned in each side of the land. Except the middle row and middle column, he cultivated grass his cows and sheeps. He has cultivated different commercial crops in the middle row cells and middle column cells so that floods will not commercial crops much.

He anticipated different incomes from each cell. He maintained those income details in a table with NXN cells. He named the table an incor His formula for total profit T from complete land = (product of income tables middle row values)+(product of income tables middle colum

When he calculated his expenditure interestingly, he found it is exactly equal to the reverse of the total profit T.

He wanted to continue the same pattern of cultivation multiple times. He needs your help in designing a program that accepts income deta cell of the income table and print the investment he made for that tenure.

Read the input from STDIN and print the output to STDOUT. Do not write arbitrary strings while reading the input or while printing, as these to the standard output.

Constraint:

1<N<= 25

N is always an odd number

Input Format:

The first line of input contains an integer, N. where N is the number of partitions he planned in each side of the land Next N lines have N integers, each separated by single white space.

Output Format:

___

```
n = int(input())
mat [] =
for i in range (n):
lis list (map (int, input ().split())) = mat.append(lis)
mid_col_pro = mat [0] [1] *mat [1] [1] *mat [2] [1]
mid_row_pro = mat [1] [0] *mat [1] [1] *mat [1] [2]
print (str (mid_col_pro+mid_row_pro) [::-1])
```

Write a program to accept two message strings, out of which one is the "Sent Message" string S, whereas the other is the "Received Message' st form of bits. This communication over a network causes some data modifications which can be seen when the sent string and received string de To probe further, they have to be checked according to two parameters:

1) The number of bits that have shuffled from their original position II) Print '0' if the number of zeros present in the sent message is equal to the number of zeros present in the received message else print '1 Illustrate the same by reading the input from STDIN and writing the output to STDOUT You should not write arbitrary strings while reading the inp to the standard output.

Constraints:

1) S and R have to be in binary format. II) 1< Lengths of S and R <= 10000

Input Format:

The first line of input has string S. The second line of input has string R.

Output Format:

The first line of output will be the number of bits that have shuffled places in the received message as compared to the sent message. The second line of output will be 1 if there is a difference in the number of zeroes and ones, if S and R are compared. Else, 0.

Sample Input 1:

10101010 11101110

Sample Output 2:

2

1

Explanation 1:

The second and sixth bit is 0 in the sent message, while they are changed to 1 in the received message. Thus the first line of output has 2 print Also, the number of zeros present in the sent message is 4 whereas in the received message it is 2 and they are not equal so 1 is printed

Sample Input 2:

10101010

10010110

Sample Output 2:

```
a=input()
b=input()
s=0
for i in range(len(a)): if a[i]!=b[i]:
s+=1
print(s)
if a.count("0")!=b.count("0"):
print (1)
else:
print (0)
```